

Rok II Semestr IV Zespół: Mateusz Tręda Karol Kuc Dominik Jarzyło Adres repozytorium: https://github.com/Domssson/IO-Volleyball	Prowadzący: dr inż. Marcin Czajkowski
--	--

1.) Zadanie

Projekt będzie systemem wspomagającym pracę oraz działanie kadry sportowej w analizie meczów.

System pozwala na rejestrację, prowadzenie oraz analizę meczy siatkarskich oraz osób w mecz zaangażowane. Zanim mecz się rozpocznie to trzeba zarezerwować halę na termin rozgrywek, ustalić drużyny i przydzielić sędziego. Koordynator rezerwuje termin wydarzenia, wynajmuje sprzęt i przydziela drużyny oraz ich trenerów. Całą rejestracją, weryfikacją, zapisem danych zajmuje się Administrator.

2.) Cel budowy systemu

Główne cele systemu:

- **Zastąpienie manualnego wprowadzania danych:** wprowadzenie uczenia maszynowego, brak konieczności ręcznego przekazywania danych systemowi.
- **Dostarczanie analizy w czasie rzeczywistym:** skrócenie czasu od zakończenia akcji do uzyskania informacji zwrotnej do minimum.
- **Zaawansowana analiza:** wykorzystanie algorytmów Computer Vision do mierzenia parametrów niemierzalnych dla ludzkiego oka.

Zakres systemu:

- **Przetwarzanie obrazu (Computer Vision):** Wykrywanie zawodników, piłki oraz linii boiska w strumieniu wideo (live lub plik).
- **Moduł klasyfikacji zagrań:** Automatyczne rozpoznawanie typów akcji (zagrywka, przyjęcie, wystawa, atak, blok, obrona) oraz ich efektu.
- **Panel analityczny (Dashboard):** Generowanie dynamicznych raportów: heatmap ataku, rozkład rozegrania, heatmap zagrywki.
- **Baza danych i Archiwum:** Przechowywanie historycznych danych o zawodnikach i meczach w celu analizy trendów sezonowych.
- **Eksport danych:** Generowanie plików kompatybilnych z formatami ligowymi (np. arkusze PDF/XLS).

Kontekst systemu:

- **Źródła danych:** Kamery IP (RTMP/RTSP), kamery stacjonarne, pliki wideo lokalne.
- **Użytkownicy:** Trenerzy (podgląd taktyczny), statystycy (weryfikacja danych AI), zawodnicy (analiza własnych błędów).
- **Otoczenie:** System może integrować się z zewnętrznymi bazami danych ligowych lub systemami transmisyjnymi (nakładki graficzne TV).

Korzyści z wdrożenia:

a) **Mierzalne (Ilościowe)**

1. **Redukcja czasu pracy:** Skrócenie czasu potrzebnego na przygotowanie pełnego raportu z meczu o 70-80% (z kilku godzin do kilkunastu minut).
2. **Zmniejszenie kosztów:** Brak konieczności zatrudniania wysokiej klasy specjalisty-statystyka w mniejszych klubach (system „obsługuje się sam”).
3. **Wzrost precyzji danych:** Wyeliminowanie błędów ludzkich (tzw. "human error") wynikających ze zmęczenia statystyka podczas długich setów.
4. **Parametry fizyczne:** Możliwość dokładnego pomiaru np. zasięgu w ataku z dokładnością do 1-2 cm.

b) **Niemierzalne (Jakościowe)**

1. **Lepsza jakość treningu:** Trenerzy mogą natychmiast korygować błędy ustawienia zawodników na podstawie heatmap.
2. **Przewaga psychologiczna:** Możliwość błyskawicznego wykrycia powtarzalnych schematów przeciwnika w trakcie trwania seta (tzw. *Real-time tactical adjustment*).
3. **Nowoczesny wizerunek klubu:** Budowanie profesjonalnej kultury opartej na danych (*Data-Driven Culture*).
4. **Łatwiejsza edukacja zawodników:** Wizualizacja błędów (overlay na wideo) jest znacznie bardziej przekonująca dla zawodnika niż sucha statystyka w tabeli.

3.) Słownik

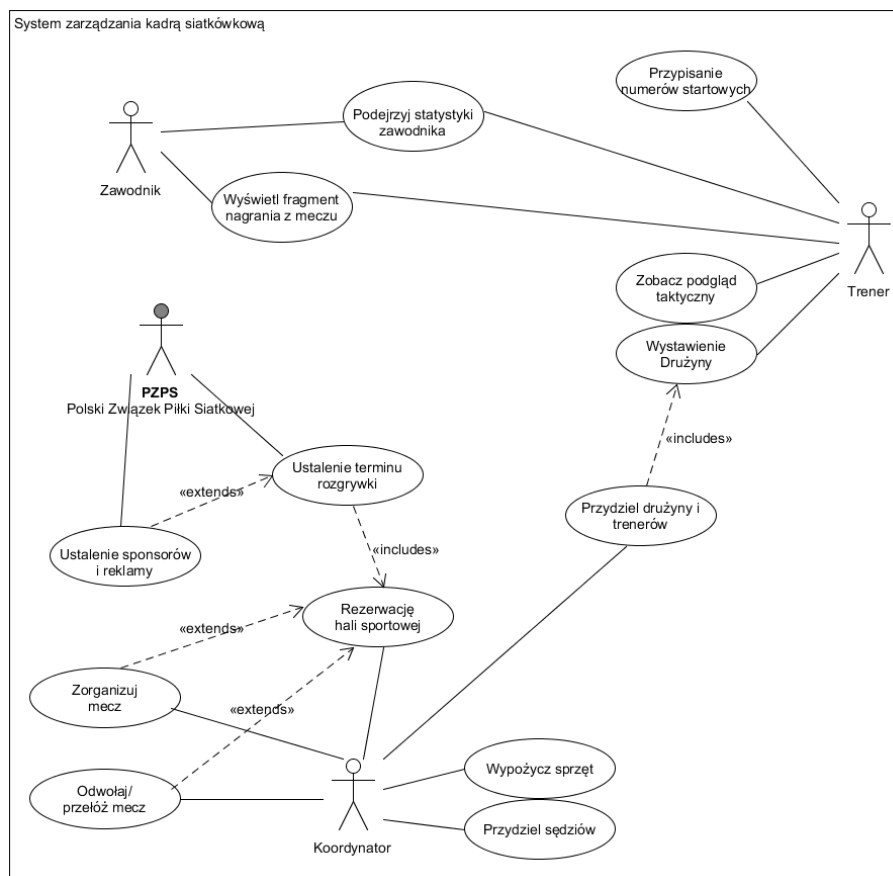
Słownik definiujący ważne, specyficzne terminy związane z dziedziną problemu, wykorzystywane w projekcie systemu analizy meczów siatkarskich.

- **System analizy meczów** – oprogramowanie wspomagające analizę przebiegu meczu siatkarskiego poprzez automatyczne przetwarzanie obrazu, identyfikację zdarzeń oraz generowanie statystyk.
- **Computer Vision (widzenie komputerowe)** – dziedzina informatyki zajmująca się analizą obrazu i wideo, wykorzystywana w systemie do wykrywania zawodników, piłki oraz elementów boiska.
- **Uczenie maszynowe (Machine Learning)** – metody pozwalające systemowi uczyć się na podstawie danych, stosowane do klasyfikacji zagrań i analizy wzorców gry.
- **Strumień wideo (Video Stream)** – ciągły przesył danych wideo z kamery (np. RTSP/RTMP), będący wejściem dla systemu analizy.
- **Detekcja obiektów (Object Detection)** – proces identyfikacji i lokalizacji obiektów (np. zawodników, piłki) w obrazie.
- **Śledzenie obiektów (Object Tracking)** – proces monitorowania ruchu wykrytych obiektów w kolejnych klatkach wideo.
- **Klasyfikacja zagrań** – automatyczne przypisywanie typu akcji (np. zagrywka, atak, blok) na podstawie analizy obrazu.
- **Akcja (Rally Event)** – pojedyncze zdarzenie w trakcie wymiany (np. atak, przyjęcie, blok).
- **Rally (wymiana)** – sekwencja akcji od momentu zagrywki do zdobycia punktu przez jedną z drużyn.
- **Heatmapa (mapa cieplna)** – wizualna reprezentacja intensywności zdarzeń (np. ataków) na boisku.
- **Dashboard (panel analityczny)** – interfejs użytkownika prezentujący dane statystyczne, wizualizacje oraz przebieg meczu w czasie rzeczywistym.
- **Analiza w czasie rzeczywistym (Real-time Analysis)** – przetwarzanie danych i prezentowanie wyników bez zauważalnego opóźnienia.
- **Overlay (nakładka wizualna)** – graficzne elementy nanoszone na obraz wideo (np. trajektoria piłki, pozycje zawodników).
- **Model AI** – wytrenowany algorytm odpowiedzialny za analizę obrazu i klasyfikację zdarzeń.
- **Metryki meczu** – wskaźniki statystyczne opisujące przebieg gry (np. skuteczność ataku, liczba błędów, liczba bloków).
- **Skuteczność ataku (Attack Efficiency)** – procentowy stosunek udanych ataków do wszystkich prób.
- **Błąd (Error)** – akcja zakończona niepowodzeniem, skutkująca punktem dla przeciwnika wynikająca z nieprawidłowego zagrania zawodnika (atak w aut, wpadnięcie w siatkę, przejście linii itp.).
- **As (Ace)** – zagrywka bezpośrednio kończąca akcję zdobyciem punktu.
- **Blok (Block)** – akcja obronna przy siatce mająca na celu zatrzymanie ataku przeciwnika.
- **Obrona (Dig)** – skuteczne podbicie piłki po ataku przeciwnika.
- **Przyjęcie (Reception)** – pierwsze odbicie piłki po zagrywce przeciwnika.

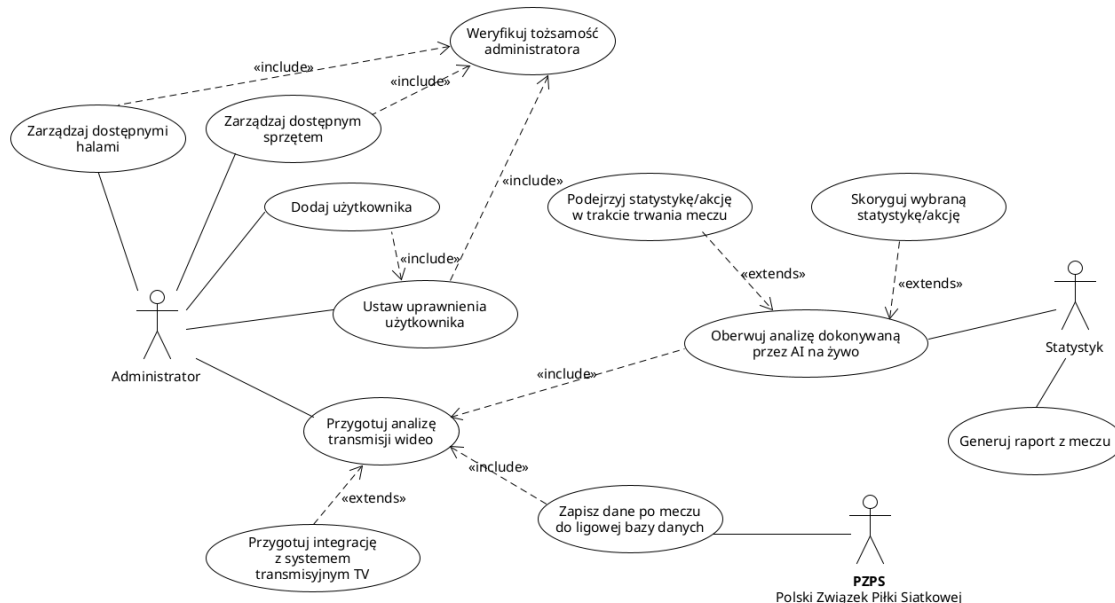
- **Przyjęcie perfekcyjne (Perfect Reception)** – przyjęcie po którym zawodnik rozgrywający dysponuje każdą opcją, przyjęcie między 0,5- 1m od siatki
- **Wystawa (Set)** – podanie piłki przez rozgrywającego do zawodnika atakującego.
- **Rotacja zawodników** – aktualne ustawienie zawodników na boisku wynikające z zasad gry.
- **Koordinator** – osoba odpowiedzialna za organizację meczu (rezerwacja hali, przydział drużyn i sędziów).
- **Administrator** – użytkownik systemu odpowiedzialny za zarządzanie danymi, użytkownikami i konfiguracją systemu.
- **Statystyk** – osoba nadzorująca poprawność danych generowanych przez system.
- **Trener** – użytkownik korzystający z danych analitycznych do podejmowania decyzji taktycznych.
- **Archiwum danych** – baza przechowująca historyczne dane o meczach i zawodnikach.
- **Eksport danych** – proces generowania raportów w formatach zewnętrznych (np. PDF, XLS).

4.) Perspektywa przypadków użycia

Diagramy przypadków użycia



Rysunek 1 Diagram przypadków użycia dla organizacji meczu



Rysunek 2 Diagram przypadków użycia dla przebiegu użycia programu w kontekście meczowym

Opisy aktorów

Termin	Definicja
Trener	Osoba posiadająca licencję trenerską, uprawniona do prowadzenia drużyny siatkówki na określonym poziomie rozgrywek a także wglądu w nagrania wideo z meczów.
Zawodnik	Osoba będąca członkiem drużyny siatkówkowej. Korzysta z systemu w celach informacyjnych – przegląda własne statystyki, ogląda fragmenty nagrań z meczów oraz śledzi wyniki rozgrywek. Najczęściej korzysta z systemu po zakończeniu meczu lub treningu.
Koordinator	Pracownik operacyjny PZPS lub wyznaczona osoba odpowiedzialna za bieżącą logistykę rozgrywek. Zarządza organizacją konkretnych meczów: rezerwuje hale sportowe, przydziela sędziów, wypożycza sprzęt sportowy oraz obsługuje sytuacje wyjątkowe, takie jak odwoływanie lub przekładanie meczów.
Statystyk	Osoba zajmująca się wyliczaniem statystyk w trakcie trwania meczu. Nadzoruje i w razie potrzeby koryguje pracę systemu analizy AI. Na koniec sporządza raport z meczu.
Administrator	Osoba uprawniona do zarządzania obiektami sportowymi i sprzętem, dodawania nowych użytkowników. Przygotowuje również system w celu rozpoczęcia analizy meczu.
PZPS	Organizacja zarządzająca rozgrywkami. Inicjuje i nadzoruje kluczowe procesy: publikowanie wyników, ustalanie terminów rozgrywek, przydzielanie drużyn i trenerów oraz pozyskiwanie sponsorów. Działa zarówno przez pracowników etatowych, jak i deleguje zadania koordinatorom.

Opisy tekstowe przypadków użycia

UC-01: Podejrzyj statystyki zawodnika

Aktorzy: Zawodnik, Trener

Ciąg podstawowy: Użytkownik (Zawodnik lub Trener) loguje się do systemu i wybiera opcję przeglądania statystyk. Wskazuje konkretnego zawodnika (lub siebie). System pobiera

z bazy danych statystyki meczowe (punkty, asysty, zagrywki, bloki, przyjęcia) i wyświetla je w formie tabelarycznej lub graficznej. Użytkownik może filtrować dane według sezonu, meczu lub kategorii statystyk.

Ciąg alternatywny / awaryjny: Jeśli zawodnik nie posiada jeszcze żadnych rozegranych meczów w systemie, wyświetlany jest komunikat „Brak danych statystycznych”. Jeśli sesja użytkownika wygasła, system przekierowuje do ekranu logowania.

Częstotliwość: Kilka–kilkanaście razy dziennie na zawodnika. Czas typowy: poniżej 3 sekund. Czas maksymalny: 10 sekund (przy dużym obciążeniu serwera).

Wartość dla aktorów: Zawodnik uzyskuje wgląd w swój własny rozwój sportowy i historię występów. Trener zyskuje narzędzie analityczne do oceny formy zawodników i podejmowania decyzji taktycznych.

UC-02: Wyświetl fragment nagrania z meczu

Aktorzy: Zawodnik, Trener (inicjatorzy)

Ciąg podstawowy: Użytkownik (Zawodnik lub Trener) wybiera mecz z listy rozegranych spotkań. System wyświetla dostępne fragmenty wideo powiązane z danym meczem. Użytkownik wybiera interesujący fragment (np. konkretny set, akcja, zagrywka). System odtwarza nagranie w odtwarzaczu wbudowanym.

Ciąg alternatywny / awaryjny: Jeśli nagranie nie zostało jeszcze przetworzone lub przesłane, system informuje: „Nagranie niedostępne – oczekiwanie na przetworzenie”. Jeśli połączenie jest zbyt wolne, system proponuje niższą jakość odtwarzania.

Częstotliwość: Kilka razy tygodniowo; spiętrzenia w trakcie przygotowań do kolejnego meczu (2–3 dni przed spotkaniem). Czas typowy: uruchomienie odtwarzacza w ciągu 5 sekund. Czas maksymalny: 20 sekund przy dużym pliku wideo.

Wartość dla aktorów: Zawodnik może analizować własne błędy i sukcesy. Trener wykorzystuje nagrania do przygotowania taktycznego drużyny i analizy gry przeciwnika.

UC-03: Ustalenie terminu rozgrywki

Aktorzy: PZPS (inicjator), Koordynator (wykonawca); powiązane «includes»: Rezerwację hali sportowej; «extends»: Ustalenie sponsorów i reklamy

Ciąg podstawowy: PZPS inicjuje proces planowania terminarza sezonu lub konkretnej kolejki rozgrywkowej. Koordynator w systemie przegląda dostępność hal sportowych i drużyn. Wybiera datę, godzinę i miejsce meczu, sprawdza konflikty terminowe. Po zatwierdzeniu system tworzy wydarzenie meczowe, automatycznie wywołując rezerwację hali (UC: Rezerwację hali sportowej) i organizację meczu (UC: Zorganizuj mecz).

Ciąg alternatywny / awaryjny: Jeśli żądana hala jest niedostępna w danym terminie, system sugeruje alternatywne obiekty lub terminy. Jeśli drużyna zgłosiła konflikt terminowy (turniej, zgrupowanie reprezentacji), system flaguje kolizję i wymaga potwierdzenia lub zmiany terminu.

Częstotliwość: Masowo na początku sezonu (planowanie całego terminarza – kilkaset meczów), a następnie kilka razy w miesiącu przy zmianach. Czas typowy: 10–15 minut na zaplanowanie jednej kolejki. Czas maksymalny: 2 godziny przy planowaniu pełnego terminarza sezonu.

Wartość dla aktorów: PZPS uzyskuje spójny, zatwierdzony terminarz rozgrywek. Koordynator ma jasny plan działań operacyjnych. Drużyny i trenerzy z wyprzedzeniem znają harmonogram.

UC-04: Ustalenie sponsorów i reklamy

Aktorzy: PZPS (jedyne akty); relacja «extends» względem Ustalenia terminu rozgrywki

Ciąg podstawowy: W trakcie lub po ustaleniu terminu rozgrywki, gdy spełnione są określone warunki (np. mecz rangi krajowej, finał), PZPS opcjonalnie uruchamia podproces pozyskiwania sponsorów. Pracownik PZPS wprowadza do systemu dane sponsorów przypisanych do danego wydarzenia: nazwę sponsora, typ reklamy (banner, spot, nazwa na koszulkach), budżet.

Ciąg alternatywny / awaryjny: Jeśli sponsor wycofa się przed meczem, system umożliwia usunięcie wpisu i powiadomienie odpowiednich służb. Przypadek nie jest wywoływany dla meczów niższych lig bez umów sponsorskich.

Częstotliwość: Kilka–kilkanaście razy w sezonie (głównie dla meczów ekstraklasy i pucharowych). Czas typowy: 20 minut. Czas maksymalny: 1 godzina.

Wartość dla aktorów: PZPS systematyzuje informacje o sponsoringu i zapewnia właściwą ekspozycję reklamową podczas meczów, co przekłada się na wywiązanie się z umów sponsorskich.

UC-05: Zorganizuj mecz

Aktorzy: Koordynator (główny wykonawca)

Ciąg podstawowy: Po ustaleniu terminu system automatycznie inicjuje ten przypadek użycia. Koordynator w systemie przechodzi przez listę kontrolną organizacji meczu: potwierdza obsadę sędziowską, sprawdza dostępność sprzętu, weryfikuje przyjazd drużyn. Oznacza kolejne punkty listy jako zrealizowane. System rejestruje status gotowości meczu.

Ciąg alternatywny / awaryjny: Jeśli jeden z elementów listy kontrolnej nie może zostać spełniony (np. choroba sędziego), system eskaluje problem do PZPS z propozycją rozwiązania. Jeśli mecz nie osiągnie statusu „gotowy do rozegrania” na 24h przed terminem, system generuje alert.

Częstotliwość: Raz na każdy zaplanowany mecz. Czas typowy: 30–60 minut (rozłożone na kilka dni). Czas maksymalny: kilka godzin w dniu meczu przy problemach organizacyjnych.

Wartość dla aktorów: Koordynator ma pełną kontrolę nad gotowością operacyjną meczu. PZPS zyskuje pewność, że spotkania są właściwie przygotowane.

UC-06: Rezerwację hali sportowej

Aktorzy: Koordynator; wywoływany przez Ustalenie terminu rozgrywki («includes»)

Ciąg podstawowy: System automatycznie inicjuje rezerwację po zatwierdzeniu terminu meczu. Koordynator w systemie wybiera halę sportową z bazy obiektów, sprawdza jej dostępność w kalendarzu i dokonuje rezerwacji na wymagany czas (zazwyczaj termin meczu + 2h na przygotowanie). System wysyła automatyczne potwierdzenie do zarządcy hali i zapisuje rezerwację.

Ciąg alternatywny / awaryjny: Jeśli hala jest już zajęta, system wyświetla listę dostępnych obiektów w tym samym mieście lub regionie. Jeśli rezerwacja zostanie anulowana przez zarządcę hali, system powiadamia Koordynatora i PZPS o konieczności znalezienia zastępczego obiektu.

Częstotliwość: Raz na każdy mecz (tyle samo co UC-06). Spiętrzenia przy planowaniu całego sezonu (jednorazowo kilkaset rezerwacji). Czas typowy: 5 minut. Czas maksymalny: 30 minut przy trudnościach z dostępnością.

Wartość dla aktorów: Koordynator ma pewność co do miejsca rozegrania meczu. PZPS zyskuje udokumentowane rezerwacje obiektów, minimalizując ryzyko kolizji.

UC-07: Przydział drużyny i trenerów

Aktorzy: Koordynator (inicjator), Trener (uczestnik); powiązane «includes»: Wystawienie Drużyny, Wyświetl fragment nagrania z meczu, Podejrzyj statystyki zawodnika, Zobacz podgląd taktyczny

Ciąg podstawowy: PZPS inicjuje przydziały na początku sezonu lub w trakcie jego trwania (np. play-offy). Pracownik PZPS w systemie przegląda listę zarejestrowanych drużyn i trenerów z aktywnymi licencjami. Przypisuje drużyny do lig i grup rozgrywkowych, a trenerów do drużyn. System weryfikuje poprawność przydziałów (aktywna licencja, brak konfliktów). Po zatwierdzeniu generowane są powiadomienia dla zainteresowanych trenerów.

Ciąg alternatywny / awaryjny: Jeśli trener nie posiada ważnej licencji, system blokuje przydział i wyświetla ostrzeżenie. Jeśli drużyna nie spełnia wymagań ligowych (np. za mała kadra), system flaguje problem przed zatwierdzeniem.

Częstotliwość: Głównie raz na sezon (sierpień–wrzesień), z drobnymi korektami w trakcie roku. Czas typowy: 1–2 godziny na przydziały dla całej ligi. Czas maksymalny: pół dnia roboczego.

Wartość dla aktorów: PZPS zapewnia kompletność i zgodność obsady ligowej z regulaminem. Trenerzy uzyskują formalne potwierdzenie prowadzenia drużyny i dostęp do narzędzi systemowych.

UC-08: Wystawienie Drużyny

Aktorzy: Trener; wywoływany przez Przydział drużyny i trenerów («includes»)

Ciąg podstawowy: Przed meczem Trener loguje się do systemu i otwiera formularz składu meczowego. Z listy zawodników swojej drużyny wybiera maksymalnie 14 graczy (zgodnie z przepisami siatkówki), wyznacza libero, kapitana i ewentualnych zmienników. Zatwierdza skład. System sprawdza poprawność formalną (liczba zawodników, ważność licencji graczy) i zapisuje skład meczowy, udostępniając go sędziom i PZPS.

Ciąg alternatywny / awaryjny: Jeśli zawodnik jest kontuzjowany (status w systemie) lub zdyskwalifikowany, system ostrzega trenera przed zatwierdzeniem składu. Jeśli skład zostanie złożony po wymaganym terminie (np. 1h przed meczem), system rejestruje opóźnienie i powiadamia koordynatora.

Częstotliwość: Przed każdym meczem drużyny, czyli kilka razy tygodniowo w sezonie. Czas typowy: 5–10 minut. Czas maksymalny: 20 minut.

Wartość dla aktorów: Trener formalizuje decyzję taktyczną dotyczącą składu. System i sędziowie uzyskują oficjalny, zatwierdzony skład meczowy.

UC-09: Zobacz podgląd taktyczny

Aktorzy: Trener; wywoływany przez Przydział drużyny i trenerów («includes»)

Ciąg podstawowy: Trener po zalogowaniu wybiera opcję analizy taktycznej. System wyświetla narzędzie do rysowania ustawień na boisku (schematyczne boisko siatkówki z pozycjami). Trener tworzy lub przegląda zapisane schematy taktyczne (rotacje, przyjęcia, ataki). Może eksportować schemat do PDF lub udostępnić zawodnikom. System zapisuje tworzone schematy w profilu drużyny.

Ciąg alternatywny / awaryjny: Jeśli trener próbuje edytować schemat drużyny, do której nie jest przypisany, system odmawia dostępu. Jeśli narzędzie rysowania nie ładuje się poprawnie (błąd JS), system proponuje wersję uproszczoną (statyczne diagramy).

Częstotliwość: Kilka razy tygodniowo, intensywniej w tygodniu poprzedzającym mecz. Czas typowy: 15–30 minut na sesję taktyczną. Czas maksymalny: brak ograniczenia czasowego.

Wartość dla aktorów: Trener zyskuje cyfrowe narzędzie do przygotowania taktycznego, zastępujące tradycyjne tablice. Zawodnicy mogą przeglądać schematy przed meczem.

UC-10: Przypisanie numerów startowych

Aktorzy: Trener

Ciąg podstawowy: Trener po wystawieniu składu meczowego otwiera moduł numeracji. System wyświetla listę wybranych zawodników z polami na numery koszulek. Trener przypisuje każdemu zawodnikowi numer (1–99, zgodnie z przepisami FIVB). System waliduje unikalność numerów w obrębie drużyny i zapisuje przypisania. Dane trafiają do protokołu meczowego.

Ciąg alternatywny / awaryjny: Jeśli zawodnik ma numer przypisany na stałe w swoim profilu, system automatycznie go podpowiada. Jeśli dwa mecze toczą się jednocześnie, system pozwala na różne numery koszulek dla tego samego zawodnika w różnych protokołach.

Częstotliwość: Przed każdym meczem, często łącznie z wystawianiem składu. Czas typowy: 3–5 minut. Czas maksymalny: 10 minut.

Wartość dla aktorów: Trener formalizuje numery startowe wymagane przez przepisy. Sędziowie i sekretarz meczu uzyskują kompletny protokół startowy.

UC-11: Przydział sędziów

Aktorzy: Koordynator

Ciąg podstawowy: Koordynator otwiera listę zaplanowanych meczów bez kompletnej obsady sędziowskiej. Dla każdego meczu wybiera z listy dostępnych i licencjonowanych sędziów (I sędzia, II sędzia, sędziowie liniowi, sekretarz). System sprawdza dostępność sędziego (brak konfliktu terminowego, aktywna licencja). Po zatwierdzeniu system wysyła powiadomienie do przydzielonych sędziów z danymi meczu.

Ciąg alternatywny / awaryjny: Jeśli żaden licencjonowany sędzia nie jest dostępny w danym terminie i regionie, system rozszerza wyszukiwanie na sąsiednie regiony lub generuje alert niedoboru kadrowego. Jeśli sędzia odmówi przydziału, Koordynator musi wybrać zastępstwo.

Częstotliwość: Kilka–kilkanaście razy tygodniowo w sezonie; masowe spiętrzenia przy ustalaniu obsad na początku sezonu. Czas typowy: 5–10 minut na jeden mecz. Czas maksymalny: 30 minut przy trudnej dostępności.

Wartość dla aktorów: Koordynator zapewnia kompletną i legalną obsadę sędziowską meczów. PZPS zyskuje pewność zgodności z regulaminem. Sędziowie uzyskują z wyprzedzeniem informację o przydzielonych im spotkaniach.

UC-12: Wypożycz sprzęt

Aktorzy: Koordynator

Ciąg podstawowy: Koordynator otwiera moduł zarządzania sprzętem. Dla planowanego meczu wybiera potrzebny ekwipunek z magazynu PZPS (siatka meczowa, słupki, piłki, tablice wynikowe, sprzęt nagłaśniający). System sprawdza dostępność sprzętu w danym terminie i lokalizacji. Koordynator potwierdza rezerwację. System rejestruje wydanie sprzętu i wymagany termin zwrotu.

Ciąg alternatywny / awaryjny: Jeśli wymagany sprzęt jest niedostępny (wypożyczony na inny mecz), system sugeruje wypożyczenie zewnętrzne lub transport z innej lokalizacji. Po terminie zwrotu system automatycznie generuje przypomnienie.

Częstotliwość: Raz na każdy mecz wymagający sprzętu z zasobów PZPS (nie wszystkie hale mają własny kompletny sprzęt). Czas typowy: 10 minut. Czas maksymalny: 30 minut.

Wartość dla aktorów: Koordynator ma kontrolę nad logistyką sprzętową. PZPS ogranicza ryzyko zagubienia lub uszkodzenia kosztownego wyposażenia.

UC-13: Odwołaj / przesun mecz

Aktorzy: Koordynator

Ciąg podstawowy: Na skutek decyzji PZPS lub siły wyższej (warunki pogodowe, awaria hali, pandemia) Koordynator otwiera zaplanowany mecz w systemie i wybiera opcję „Odwołaj” lub „Przesun”. W przypadku przesunięcia wprowadza nowy termin i lokalizację. System automatycznie anuluje powiązane rezerwacje (hala, sędziowie, sprzęt), generuje powiadomienia do drużyn, trenerów i sędziów oraz aktualizuje terminarz.

Ciąg alternatywny / awaryjny: Jeśli nowy termin koliduje z innymi zaplanowanymi meczami na tej hali, system informuje Koordynatora i wymaga wyboru alternatywnego obiektu. Jeśli mecz jest odwoływany mniej niż 48h przed terminem, system generuje alert o możliwych karach finansowych zgodnie z regulaminem.

Częstotliwość: Rzadko w normalnych warunkach, masowo w sytuacjach kryzysowych. Czas typowy: 15 minut. Czas maksymalny: 1 godzina

Wartość dla aktorów: Koordynator dokumentuje decyzję organizacyjną w systemie. Wszystkie zainteresowane strony (drużyny, sędziowie, hale) są natychmiastowo i automatycznie powiadamiane.

UC-14: Zarządzaj dostępnymi halami

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera w menu głównym panelu administracyjnego opcję zarządzania halami. Wyświetlana jest lista wszystkich zarejestrowanych obecnie hal, wraz z możliwością wyszukiwania oraz przycisk dodania nowej hali. Użytkownik wybiera interesującą go opcję dodania nowej hali, zaktualizowania informacji o jednej hali z listy bądź usunięcia wybranej hali. Użytkownik wypełnia odpowiedni formularz z danymi na temat nowej/wybranej hali. Zostaje wykonana dodatkowa weryfikacja tożsamości administratora (przypadek użycia „Weryfikuj tożsamość administratora”). Dane zostają zaktualizowane w bazie danych.

Ciąg alternatywny / awaryjny: Użytkownik w pewnym momencie anulował wprowadzanie danych – następuje powrót do poprzedniego menu. Weryfikacja uprawnień administratora nie powiodła się – wyświetlony zostaje komunikat błędu.

Częstotliwość: Kilka razy do roku. Dosyć rzadkie spiętrzenia, tylko przy znaczących zmianach w infrastrukturze. Czas typowy: od kilku do kilkunastu minut. Czas maksymalny: 30 minut

Wartość dla aktorów: Wyświetlany jest komunikat o prawidłowym wykonaniu operacji, do systemu zostaje dodana nowa hala lub informacje o niej zostają zaktualizowane.

UC-15: Zarządzaj dostępnym sprzętem

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera w menu głównym panelu administracyjnego opcję zarządzania sprzętem. Użytkownik wybiera halę, w której sprzęt jest/będzie dostępny. Wyświetlana jest lista sprzętu dostępnego w danej hali, wraz z możliwością wyszukiwania oraz przycisk dodania nowego sprzętu. Użytkownik wybiera interesującą go opcję dodania nowego sprzętu, zaktualizowania informacji o jednym ze sprzętów z listy bądź usunięcia go. Użytkownik wypełnia odpowiedni formularz z danymi na temat nowego/wybranego sprzętu. Zostaje wykonana dodatkowa weryfikacja tożsamości administratora (przypadek użycia „Weryfikuj tożsamość administratora”). Dane zostają zaktualizowane w bazie danych.

Ciąg alternatywny / awaryjny: Użytkownik w pewnym momencie anulował wprowadzanie danych – następuje powrót do poprzedniego menu. Weryfikacja uprawnień administratora nie powiodła się – wyświetlony zostaje komunikat błędu.

Częstotliwość: Kilka razy w miesiącu. Spiętrzenia przy masowych inwestycjach w sprzęt bądź wymianie go na nowszy. Czas typowy: od kilku do kilkadziesiąt minut. Czas maksymalny: do kilku godzin

Wartość dla aktorów: Wyświetlany jest komunikat o prawidłowym wykonaniu operacji, do systemu zostaje dodany nowy sprzęt lub informacje o nim zostają zaktualizowane.

UC-16: Dodaj użytkownika

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera w menu głównym panelu administracyjnego opcję dodania nowego użytkownika. Wypełnia formularz z danymi nowego użytkownika. Administrator ustawia uprawnienia nowego użytkownika (przypadek użycia „Ustaw uprawnienia użytkownika”) i zostaje wykonana dodatkowa weryfikacja tożsamości administratora. Dane zostają zaktualizowane w bazie danych.

Ciąg alternatywny / awaryjny: Administrator w pewnym momencie anulował wprowadzanie danych – następuje powrót do poprzedniego menu. Weryfikacja uprawnień administratora nie powiodła się – wyświetlony zostaje komunikat błędu.

Częstotliwość: Od kilku do kilkunastu razy w miesiącu. Spiętrzenia przy rejestracji nowych zespołów, przy zatrudnianiu dużej ilości nowej kadry. Czas typowy: kilkanaście minut. Czas maksymalny: kilkadziesiąt minut

Wartość dla aktorów: Do systemu zostaje dodany nowy użytkownik.

UC-17: Ustaw uprawnienia użytkownika

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera w menu głównym panelu administracyjnego opcję ustawienia uprawnień użytkownika. Wyświetlona zostaje lista użytkowników z możliwością wyszukiwania, skąd wybierany jest modyfikowany użytkownik, bądź administrator zostaje tu przekierowany z przypadku użycia „Dodaj użytkownika”. Administrator ustawia nową rolę użytkownika (administrator, koordynator, statystyk, trener, zawodnik). Zostaje wykonana dodatkowa weryfikacja tożsamości administratora (przypadek użycia „Weryfikuj tożsamość administratora”). Dane zostają zaktualizowane w bazie danych.

Ciąg alternatywny / awaryjny: Administrator w pewnym momencie anulował wprowadzanie danych – następuje powrót do poprzedniego menu. Weryfikacja uprawnień administratora nie powiodła się – wyświetlony zostaje komunikat błędu.

Częstotliwość: Od kilku do kilkunastu razy w miesiącu. Spiętrzenia przy rejestracji nowych zespołów, przy zatrudnianiu dużej ilości nowej kadry. Czas typowy: kilka minut. Czas maksymalny: 10 minut

Wartość dla aktorów: Rola użytkownika zostaje zmieniona, wyświetlana jest informacja o prawidłowym przeprowadzeniu operacji.

UC-18: Weryfikuj tożsamość administratora

Aktorzy: Administrator

Ciąg podstawowy: Wyświetlony zostaje ekran weryfikacji tożsamości administratora. Administrator wprowadza swoje specjalne ID/hasło, skanuje swoją kartę.

Ciąg alternatywny / awaryjny: Zostają wprowadzone nieprawidłowe dane – zostaje zgłoszony błąd.

Częstotliwość: Do kilku razy w tygodniu. Spiętrzenia przy masowych inwestycjach, zmianach kadry. Czas typowy: do jednej minuty. Czas maksymalny: do dziesięciu minut

Wartość dla aktorów: Tożsamość administratora zostaje zweryfikowana i kontynuowane jest wykonywanie akcji wymagającej danych uprawnień.

UC-19: Przygotuj analizę transmisji wideo

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera przycisk rozpoczęcia nowego meczu. Do systemu zostają kolejno podpięte i skonfigurowane kamery bądź pliki wideo. Następuje kalibracja i weryfikacja, czy system działa poprawnie. Wypełniany jest protokół z informacjami dotyczącymi meczu, który ma się odbyć. Tuż przed rozpoczęciem meczu administrator naciska przycisk rozpoczynający analizę.

Ciąg alternatywny / awaryjny: Następuje błąd podczas konfiguracji kamer – po rozwiązaniu problemu, administrator ponawia weryfikację działania systemu.

Częstotliwość: Co 1-kilka dni. Spiętrzenia w środku sezonu. Czas typowy: kilka godzin. Czas maksymalny: do pięciu godzin

Wartość dla aktorów: Do systemu zostają podłączone kamery, rozpoczyna się analiza obrazu z kamer przez AI, otwierany jest główny interfejs analizy meczu.

UC-20: Przygotuj integrację z systemem transmisyjnym TV

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera opcję przekierowania wideo z kamer do analizy w celu transmisji TV. Wybierana i konfigurowana jest wybrana nakładka prezentująca wyniki meczu. Pracownik telewizji bądź serwisu online podpina i konfiguruje sprzęt do transmisji.

Ciąg alternatywny / awaryjny: Następuje błąd podczas konfiguracji systemu transmisyjnego – administrator i pracownik telewizji bądź serwisu online rozwiązują problem i kontynuują konfigurację.

Częstotliwość: Co kilka dni. Spiętrzenia w środku sezonu. Czas typowy: kilkadziesiąt minut. Czas maksymalny: do pięciu godzin

Wartość dla aktorów: Rozpoczyna się przekazywanie transmisji na żywo z nałożoną odpowiednią nakładką do stacji telewizyjnej lub serwisu online.

UC-21: Zapisz dane po meczu do ligowej bazy danych

Aktorzy: Administrator, PZPS (Polski Związek Piłki Siatkowej)

Ciąg podstawowy: Po zakończeniu meczu wyświetlane jest jego podsumowanie. Dokonywana jest dodatkowa weryfikacja zebranych statystyk. Administrator zatwierdza zebrane dane, które zostają następnie przesłane do ligowej bazy danych.

Ciąg alternatywny / awaryjny: –

Częstotliwość: Co 1-kilka dni. Spiętrzenia w środku sezonu. Czas typowy: kilkadziesiąt minut. Czas maksymalny: do kilku godzin

Wartość dla aktorów: Do ligowej bazy danych zostają wpisane informacje na temat meczu zebrane w trakcie działania systemu.

UC-22: Obserwuj analizę dokonywaną przez AI na żywo

Aktorzy: Statystyk

Ciąg podstawowy: Statystyk potwierdza swoją tożsamość na urządzeniu, na którym będzie weryfikował dane. Po rozpoczęciu meczu wyświetlany jest panel główny analizy meczu wraz z m.in. heatmapą, listą zarejestrowanych akcji i podstawowymi statystykami. Statystyk obserwuje i weryfikuje poprawność pracy systemu.

Ciąg alternatywny / awaryjny: –

Częstotliwość: Co 1-kilka dni. Spiętrzenia w środku sezonu. Czas typowy: 90-120 minut. Czas maksymalny: kilka godzin

Wartość dla aktorów: Statystyk widzi wizualizację pracy systemu i zbierane przez system w czasie rzeczywistym dane.

UC-23: Generuj raport z meczu

Aktorzy: Statystyk

Ciąg podstawowy: Po zakończeniu meczu zostaje wyświetlone menu podsumowania meczu. Statystyk jeszcze raz weryfikuje integralność zebranych danych i statystyk. Statystyk wybiera przycisk wygenerowania raportu. Zostaje wyświetlony formularz do raportu, statystyk uzupełnia brakujące dane i podaje opis meczu. Po naciśnięciu przycisku zatwierdzającego zostaje wygenerowany gotowy do pobrania raport w formacie PDF.

Ciąg alternatywny / awaryjny: –

Częstotliwość: Co 1-kilka dni. Spiętrzenia w środku sezonu. Czas typowy: kilkadziesiąt minut. Czas maksymalny: kilka godzin

Wartość dla aktorów: Raport z meczu z najważniejszymi statystykami i ogólnym opisem w formacie PDF.

UC-24: Podejrzuj statystykę/akcję w trakcie trwania meczu

Aktorzy: Statystyk

Ciąg podstawowy: Statystyk podczas analizy meczu naciska przycisk wyświetlający szczegóły na temat danej statystyki/akcji. Ukazuje się okno pokazujące szczegóły na temat danej statystyki/akcji, jej ostatnich zmian wraz z ich powodami. Dostępne są również przyciski służące do powrotu oraz do korekcji statystyki.

Ciąg alternatywny / awaryjny: –

Częstotliwość: Kilkadziesiąt razy w trakcie jednego meczu. Spiętrzenia w trakcie meczu. Czas typowy: ok. minuty. Czas maksymalny: do pięciu minut

Wartość dla aktorów: Wyświetlone zostają szczegółowe dane pokazujące zmiany danej statystyki w czasie i akcje, które wpłynęły na jej zmianę.

UC-25: Skoryguj wybraną statystykę/akcję

Aktorzy: Statystyk

Ciąg podstawowy: Statystyk podczas analizy meczu naciska przycisk wyświetlający szczegóły na temat danej statystyki/akcji. Ukazuje się okno pokazujące szczegóły na temat danej statystyki/akcji, jej ostatnich zmian wraz z ich powodami. Statystyk wybiera przycisk umożliwiający jej korektę. Statystyk wprowadza w wyznaczonym miejscu prawidłową wartość statystyki bądź zmienia typ rozpoznanej akcji, po czym zapisuje zmiany. Następuje powrót do panelu głównego analizy.

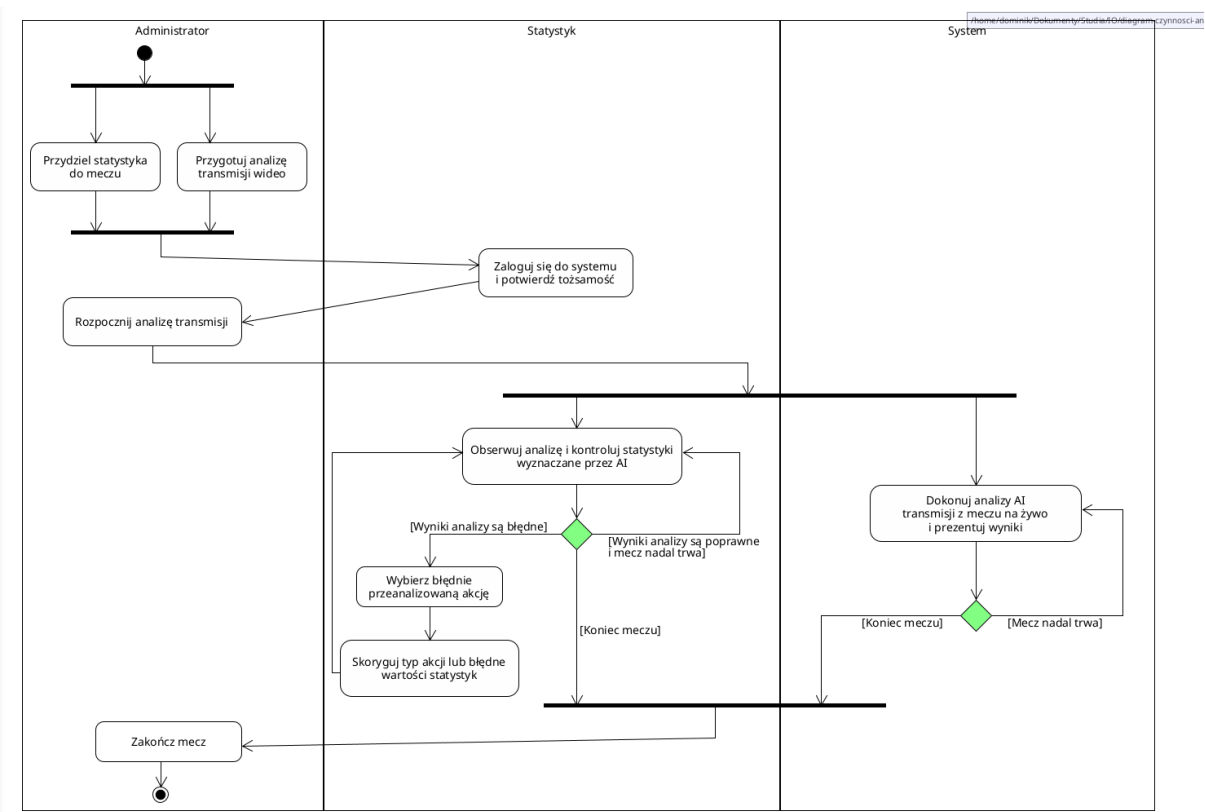
Ciąg alternatywny / awaryjny: –

Częstotliwość: Do kilkunastu razy w trakcie jednego meczu. Spiętrzenia w trakcie meczu. Czas typowy: ok. minuty. Czas maksymalny: do pięciu minut

Wartość dla aktorów: Wartość wybranej statystyki lub rodzaj akcji zostają skorygowane po błędnym automatycznym rozpoznaniu.

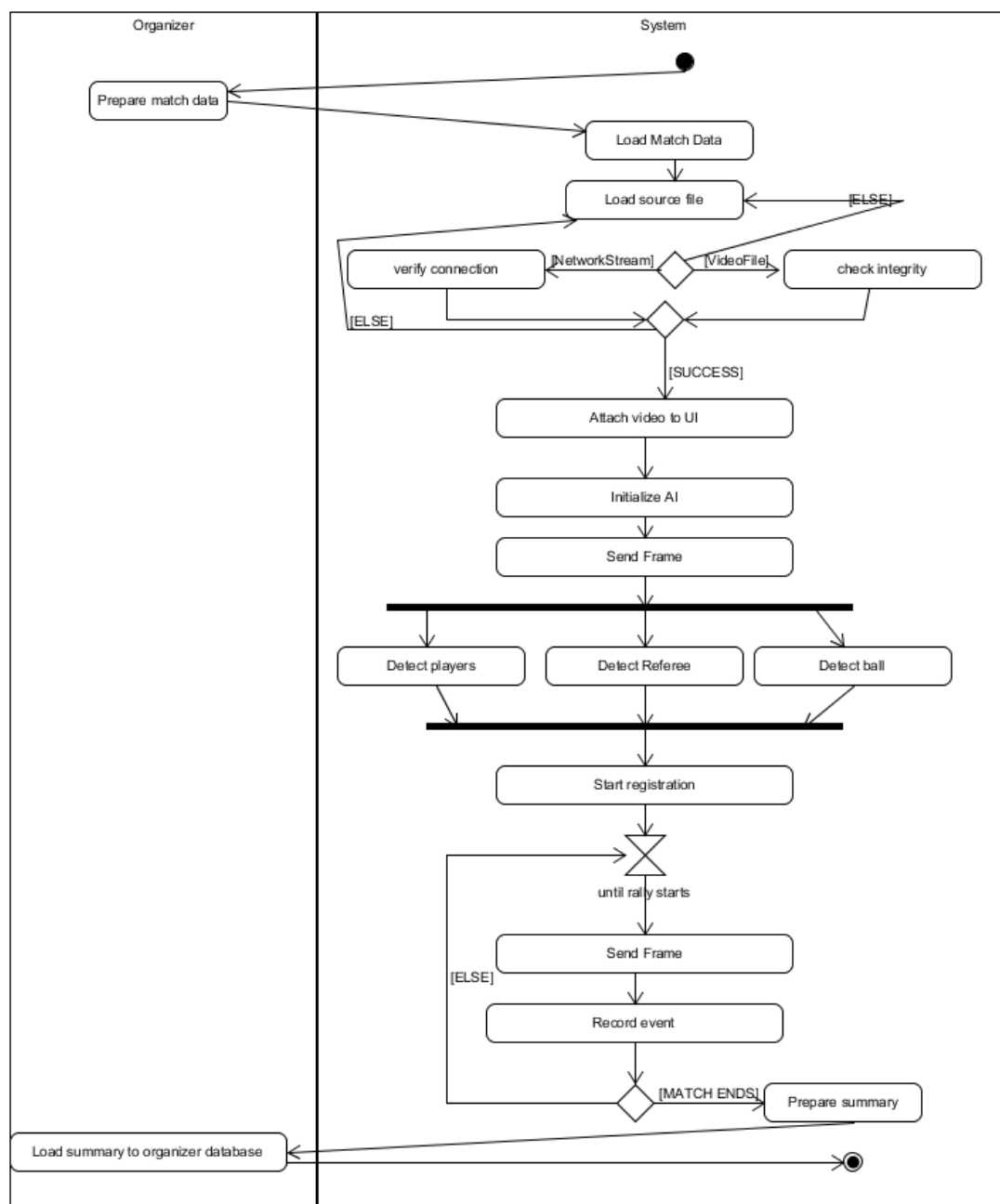
Diagramy czynności

Dominik Jarzyło



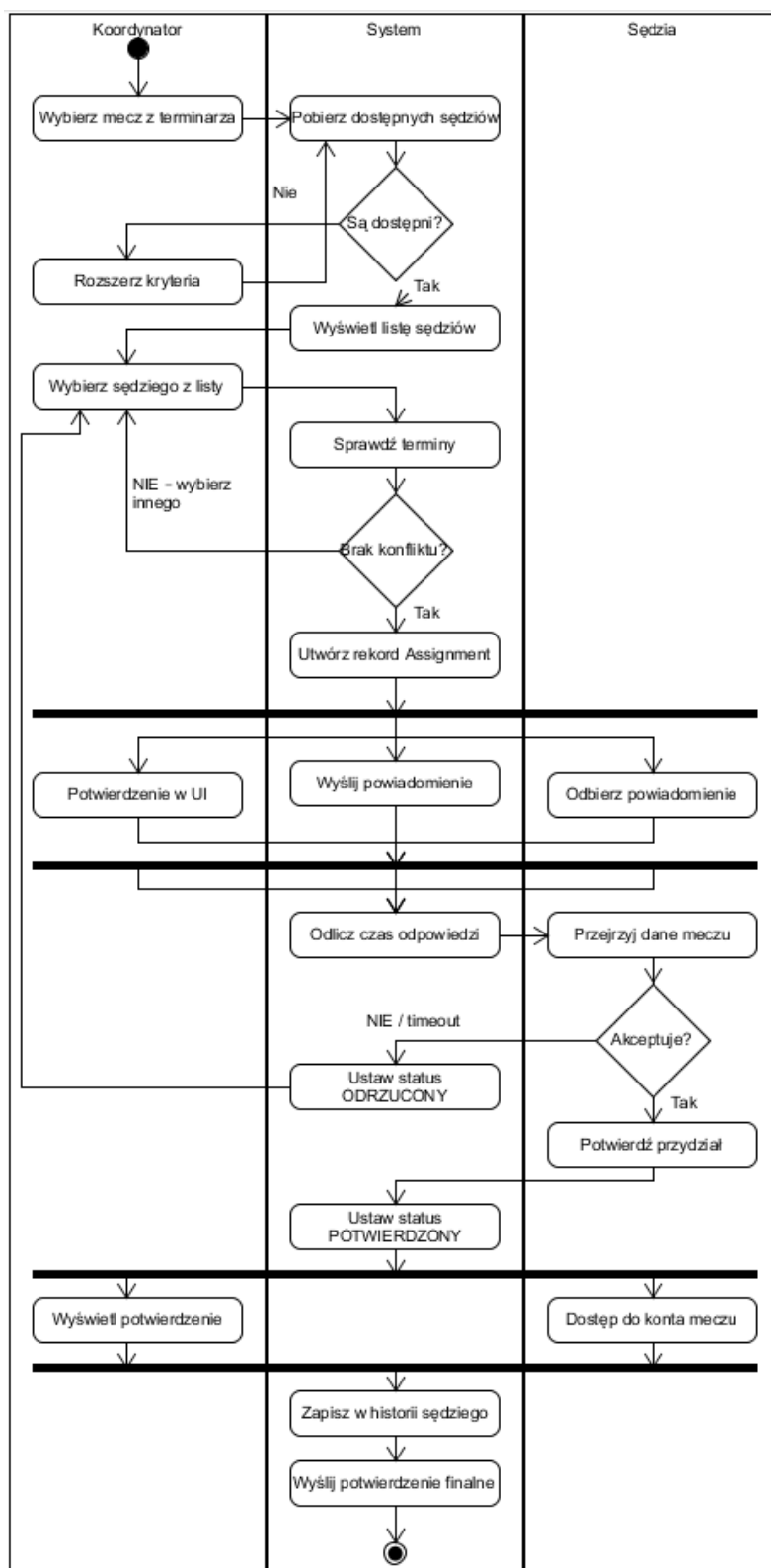
Rysunek 3 Diagram czynności dla przypadku użycia „Obserwuj analizę dokonywaną przez AI na żywo”

Karol Kuc



Rysunek 4 Diagram czynności dla przypadku użycia "Przygotuj analizę transmisji wideo"

Mateusz Tręda



Rysunek 5 Diagram czynności dla przypadku użycia „Przydziel sędziów”

Diagram jest podzielony na trzy partycje, Koordynator, System i Sędzia. Ścieżka „System” zawiera sekwencję działań i punktów decyzyjnych. Wpiew Koordynator wybiera mecz, do

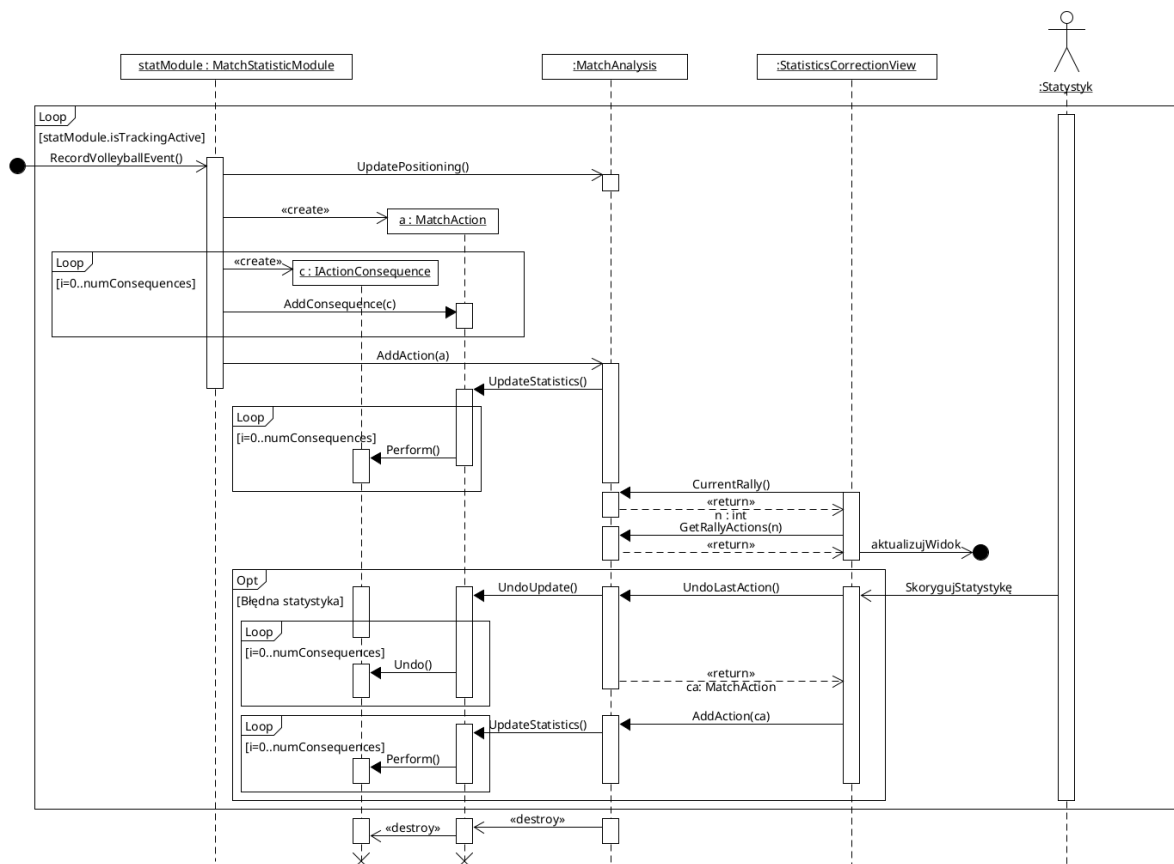
którego ma być przydzielony sędzia. Później Koordynator wykonuje request pobrania listy sędziów na podstawie swoich kryteriów. Jeśli są dostępni to może ich wybrać z listy sprawdzając konflikty z terminami.

Po wybraniu sędziego tworzy się obiekt Assignment czyli Przydział sędziego. Do sędziego jest wysyłane powiadomienie o przydziale. Operacja jest widoczna dla wszystkich 3 partycji. Sędzia ma dostęp do danych meczu, gdzie jest przydzielany i ma możliwość akceptować lub odrzucić. Status Przydziału jest potwierdzony a sędzia otrzymuje dostęp do konta analizy meczu.

Na końcu wszystko jest zapisane w historii i jest wykonane finalne potwierdzenie w Emailu sędziego.

Diagramy przebiegu

Dominik Jarzyło

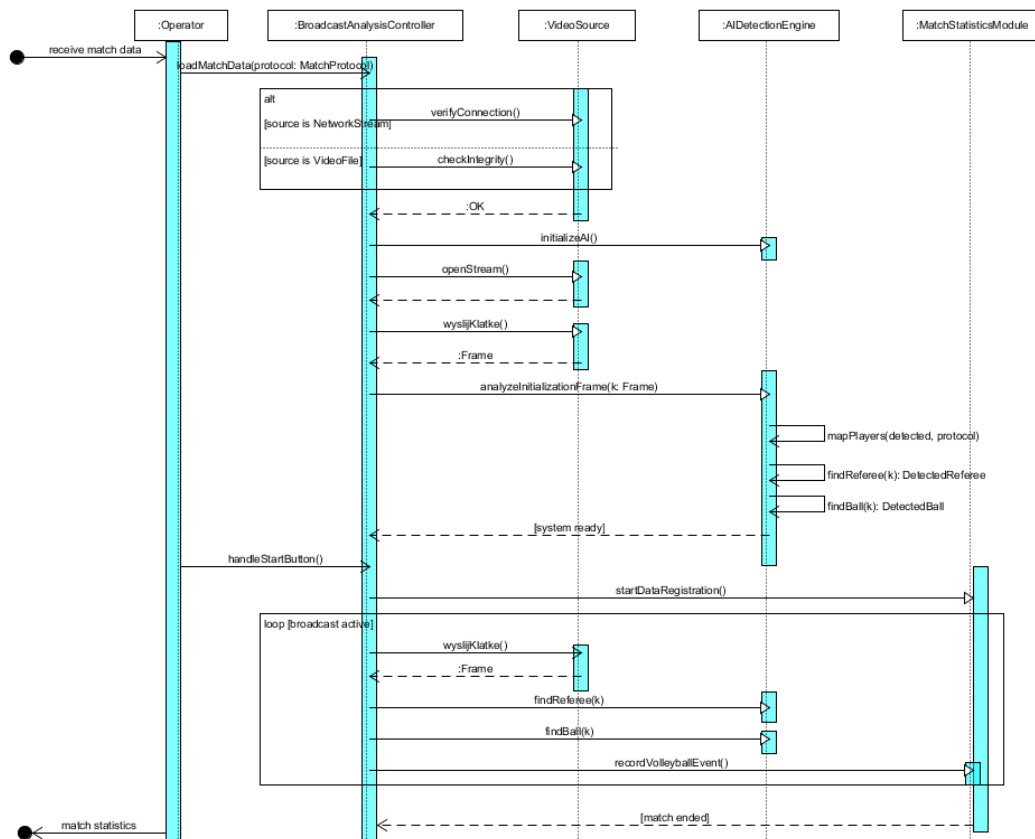


Rysunek 6 Diagram przebiegu dla przypadku użycia „Obserwuj analizę dokonywaną przez AI na żywo”

Komunikaty odbierane i wysyłane na zewnątrz fragmentu systemu:

- RecordVolleyballEvent() – wysyłany do obiektu klasy MatchStatisticsModule, gdy system analizy AI zidentyfikuje pewne wydarzenie w celu dalszej analizy jego rodzaju oraz wpływu na statystyki,
- AktualizujWidok – komunikat powodujący zaktualizowanie danych wyświetlanych przez interfejs użytkownika,
- SkorygujStatystykę – komunikat wysyłany przez interfejs użytkownika do obiektu klasy StatisticsCorrectionView, gdy statystyk dokona korekcji pewnej akcji i związanych z nią statystyk. Powoduje to cofnięcie w systemie ostatnio rozpoznanej akcji, zmianę odpowiednich parametrów w obiekcie akcji i zarejestrowanie jej ponownie.

Karol Kuc



Rysunek 7 Diagram przebiegu dla przypadku użycia „Przygotuj analizę transmisji wideo”

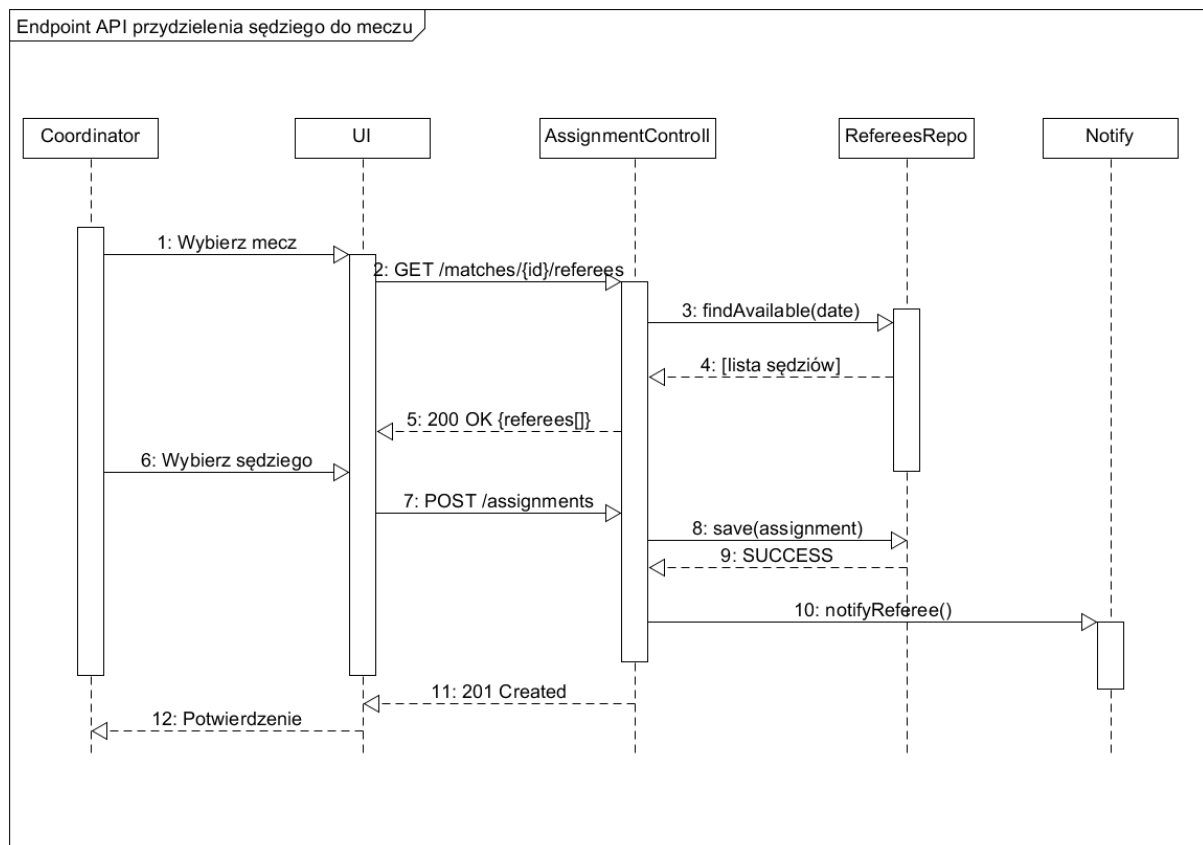
Poniższy diagram przebiegu przedstawia sekwencję komunikatów wymienianych między obiektami systemu podczas realizacji zdarzenia systemowego Przygotuj analizę transmisji wideo. Scenariusz obejmuje fazę inicjalizacji systemu, kalibrację silnika detekcji AI oraz rejestrację zdarzeń meczowych w trakcie transmisji.

Opis sekwencji komunikatów

1. **receive match data** → **Operator**: Zewnętrzny wyzwalacz (oznaczony stanem początkowym na diagramie) informuje operatora o dostępności danych meczowych do przetworzenia.
2. **Operator** → **:BroadcastAnalysisController**: `loadMatchData(protocol: MatchProtocol)` — Operator inicjuje scenariusz, przekazując do kontrolera obiekt protokołu meczowego zawierający identyfikator meczu, datę, nazwę areny, składy drużyn oraz listę sędziów.
3. **Fragment *alt*** [typ źródła wideo] — Kontroler weryfikuje typ podłączonego źródła wideo. Możliwe są dwa warianty:
 - [source is NetworkStream]: **:BroadcastAnalysisController** → **:VideoSource**: `verifyConnection()` — asynchroniczne wywołanie weryfikujące dostępność połączenia sieciowego ze strumieniem.
 - [source is VideoFile]: **:BroadcastAnalysisController** → **:VideoSource**: `checkIntegrity()` — asynchroniczne wywołanie sprawdzające spójność i dostępność pliku wideo.
4. **:VideoSource** → **:BroadcastAnalysisController**: `:OK` — Obiekt źródła wideo zwraca komunikat zwrotny potwierdzający pomyślną weryfikację. Kontroler może przejść do kolejnego etapu.
5. **:BroadcastAnalysisController** → **:AIDetectionEngine**: `initializeAI()` — Kontroler asynchronicznie inicjalizuje silnik detekcji AI, ładując modele detekcji
6. **:BroadcastAnalysisController** → **:VideoSource**: `openStream()` — Kontroler asynchronicznie poleca obiektowi źródła wideo otwarcie strumienia/pliku. Źródło wideo potwierdza gotowość komunikatem zwrotnym (linia przerywana).
7. **:BroadcastAnalysisController** → **:VideoSource**: `wyslijKlatke()` — Kontroler żąda przesłania klatki inicjalizacyjnej.
8. **:VideoSource** → **:BroadcastAnalysisController**: `:Frame` — Obiekt źródła wideo zwraca klatkę wideo (obiekt klasy `Frame`) zawierającą dane obrazu z rozdzielczością i znacznikiem czasu. Klatka ta posłuży do kalibracji silnika AI.
9. **:BroadcastAnalysisController** → **:AIDetectionEngine**: `analizeInitializationFrame(k: Frame)` — Kontroler asynchronicznie zleca silnikowi AI analizę klatki inicjalizacyjnej. Klatka jest przekazywana jako argument wywołania.
10. **Wywołania własne :AIDetectionEngine** — Silnik detekcji wykonuje trzy wewnętrzne operacje na klatce inicjalizacyjnej:
 - `mapPlayers(detected, protocol)` — mapuje wykrytych zawodników na rekordy z protokołu meczowego na podstawie numerów koszulek.
 - `findReferee(k): DetectedReferee` — lokalizuje sędziego na klatce i zwraca obiekt `DetectedReferee` z ramką detekcji.
 - `findBall(k): DetectedBall` — lokalizuje piłkę na klatce i zwraca obiekt `DetectedBall` z ramką detekcji.
11. **:AIDetectionEngine** → **:BroadcastAnalysisController**: `[system ready]` — Silnik AI zwraca komunikat przerywaną linią sygnalizujący ukończenie kalibracji. System jest gotowy do rejestracji meczu.

12. **Operator** → **:BroadcastAnalysisController:** handleStartButton() — Operator ręcznie naciska przycisk START, zlecając kontrolerowi uruchomienie rejestracji danych statystycznych.
13. **:BroadcastAnalysisController** → **:MatchStatisticsModule:** startDataRegistration() — Kontroler asynchronicznie uruchamia moduł statystyk. Od tego momentu system wchodzi w fazę właściwej analizy transmisji.

Mateusz Tręda



Uczestnicy (linie życia)

Koordynator — aktor ludzki inicjujący cały proces. Podejmuje decyzje: wybiera mecz i wskazuje konkretnego sędziego z otrzymanej listy. Jest pierwszym i ostatnim ogniwem przepływu.

UI — warstwa prezentacji (interfejs webowy/aplikacja). Pośredniczy między działaniami koordynatora a logiką backendową. Tłumaczy kliknięcia użytkownika na wywołania HTTP i prezentuje odpowiedzi systemu.

AssignmentControll — kontroler po stronie serwera (backend REST API). Główny orkiestrator logiki biznesowej: przyjmuje żądania HTTP, koordynuje pracę repozytorium i serwisu powiadomień, zwraca odpowiedzi do UI.

RefereesRepository (RefereeRepo) — warstwa dostępu do danych odpowiedzialna za sędziów. Odpytuje bazę danych w celu znalezienia dostępnych sędziów oraz utrwalenia nowego przydziału. Nie zawiera logiki biznesowej — wyłącznie operacje na danych.

NotificationService (Notify) — serwis odpowiedzialny za wysyłanie powiadomień. Wywoływany asynchronicznie po zapisaniu przydziału. Informuje sędziego o nowym zadaniu (e-mail / SMS).

Chronologiczny opis akcji

Akcja 1 — Koordynator → UI: „Wybierz mecz” Koordynator klika wybrany mecz w interfejsie. Jest to jedyne działanie czysto ludzkie inicjujące cały łańcuch. UI rejestruje zdarzenie i przechodzi do kroku 2.

Akcja 2 — UI → AssignmentController: GET /matches/{id}/referees UI wysyła żądanie HTTP GET do kontrolera, przekazując identyfikator wybranego meczu w ścieżce URL. Żądanie pyta: „kto jest dostępny do sędziowania tego meczu?”

Akcja 3 — AssignmentControll → RefereeRepo: findAvailable(date) Kontroler deleguje zapytanie do repozytorium, przekazując datę meczu jako parametr filtrowania. Repozytorium przeszukuje bazę danych pod kątem sędziów z aktywną licencją, bez konfliktu terminowego w tym dniu i z odpowiednim regionem.

Akcja 4 — JudgeRepo → AssignmentController: [lista sędziów] Repozytorium zwraca wynik zapytania — kolekcję obiektów sędziów spełniających kryteria. Komunikat oznaczony nawiasami kwadratowymi sygnalizuje, że jest to odpowiedź zwrotna (return), nie nowe wywołanie. Linia przerywana potwierdza charakter odpowiedzi.

Akcja 5 — AssignmentController → UI: 200 OK {judges[]} Kontroler odsyła do UI odpowiedź HTTP 200 z listą dostępnych sędziów w ciele odpowiedzi (JSON). UI renderuje listę — koordynator widzi kandydatów do wyboru.

Akcja 6 — Koordynator → UI: „Wybierz sędziego” Drugie i ostatnie działanie ludzkie w procesie. Koordynator wskazuje konkretną osobę z wyświetlonej listy i zatwierdza wybór. UI rejestruje decyzję i inicjuje zapis przydziału.

Akcja 7 — UI → AssignmentController: POST /assignments UI wysyła żądanie HTTP POST do kontrolera z danymi nowego przydziału w ciele żądania (identyfikator meczu + identyfikator sędziego). Użycie metody POST sygnalizuje tworzenie nowego zasobu.

Akcja 8 — AssignmentController → JudgeRepo: save(assignment) Kontroler przekazuje gotowy obiekt przydziału do repozytorium w celu utrwalenia go w bazie danych. Repozytorium wykonuje operację INSERT i zwraca potwierdzenie.

Akcja 9 — JudgeRepo → AssignmentController: SUCCESS Repozytorium potwierdza pomyślny zapis — przydział istnieje teraz w bazie danych ze statusem OCZEKUJĄCY. Linia przerywana oznacza komunikat zwrotny.

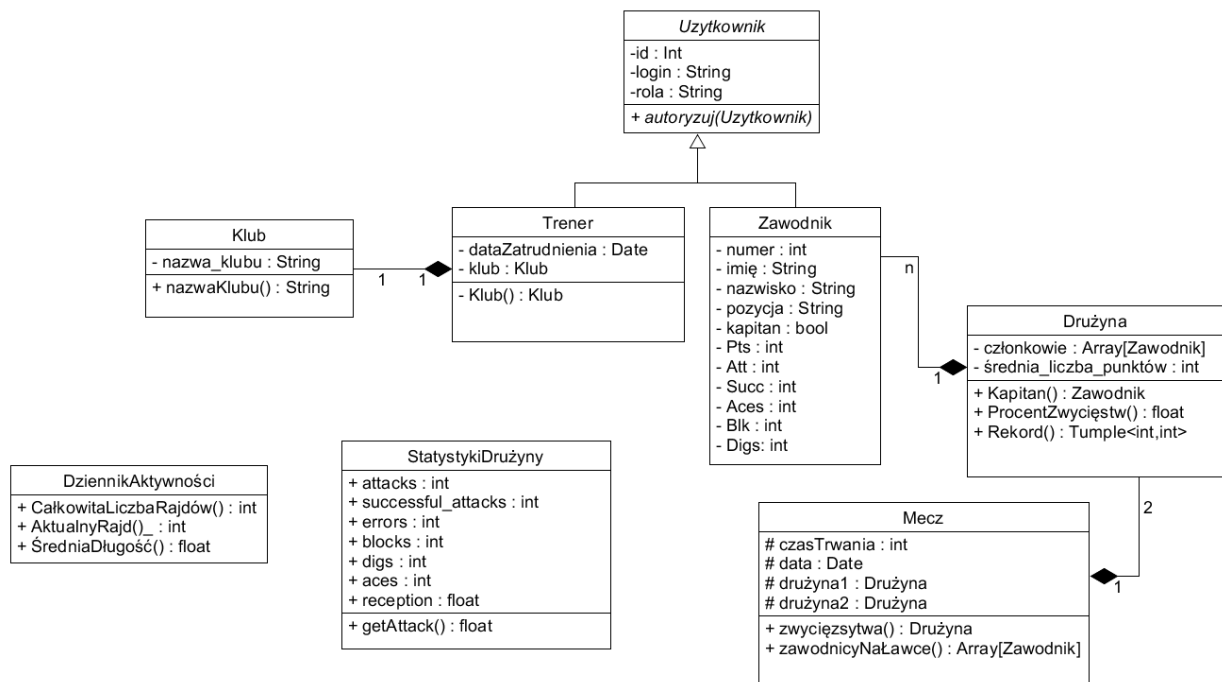
Akcja 10 — AssignmentController → NotificationService: notifyJudge() Po pomyślnym zapisie kontroler wywołuje serwis powiadomień. To wywołanie jest logicznie asynchroniczne — kontroler nie czeka na zakończenie wysyłki przed zwróceniem odpowiedzi do UI. Sędzia otrzymuje wiadomość e-mail lub SMS z danymi meczu.

Akcja 11 — AssignmentController → UI: 201 Created Kontroler zwraca do UI odpowiedź HTTP 201, sygnalizując pomyślne utworzenie nowego zasobu (przydziału). Odpowiedź może zawierać w nagłówku Location adres URL nowo powstałego rekordu.

Akcja 12 — UI → Koordynator: „Potwierdzenie” UI wyświetla koordynatorowi komunikat sukcesu — przydział został zapisany, sędzia powiadomiony. Proces dobiegł końca. Linia przerywana wskazuje, że jest to komunikat zwrotny kończący aktywację koordynatora.

5.) Perspektywa projektowa

Mateusz Tręda



Rysunek 9 Diagram Klas dla Interfejsu użytkownika przeglądu statystyk meczu

Klasa: DziennikAktywnosci

- Klasa zbierająca informacje aktywności interfejsu
 - CałkowitaLiczbaRajdów() : int – serię udanych akcji jednej drużyny
 - AktualnyRajd() : int – zwraca długość aktualnie trwającego rajdu punktowego jednej z drużyn.
 - ŚredniaDługość() : float – zwraca średnią długość wszystkich zarejestrowanych rajdów w meczu.

Klasa: Klub

- Klasa reprezentująca klub sportowy uczestniczący w rozgrywkach.
 - nazwa_klubu : String – pełna nazwa klubu sportowego
 - nazwaKlubu() : String – zwraca nazwę klubu.

Klasa: Mecz

- Klasa reprezentująca mecz koszykówki wraz z informacjami o uczestniczących drużynach, czasie trwania oraz dacie rozegrania spotkania.
 - czasTrwania : int – czas trwania meczu wyrażony w minutach.
 - data : Date – data rozegrania meczu.
 - drużyna1 : Drużyna – pierwsza drużyna uczestnicząca w meczu.
 - drużyna2 : Drużyna – druga drużyna uczestnicząca w meczu.
 - zwycięstwo() : Drużyna – zwraca drużynę, która wygrała mecz.

6. zawodnicyNaŁawce() : Array[Zawodnik] – zwraca listę zawodników rezerwowych znajdujących się na ławce podczas meczu.

Klasa: **StatystykiDrużyny**

- Klasa reprezentująca statystyki drużyny konkretnego meczu.
 1. attacks: int – liczba zagranych ataków przez drużynę
 2. errors: int – liczba fauli/błędów.
 3. successfull_attacks: int – liczba ataków przynosząca punkt.
 4. digs: int – liczba podań.
 5. blocks: int – liczba zablokowań piłki od przeciwnika.
 6. aces: int – liczba zagrań solo przynosząca punkt.
 7. reception: int – liczba przyjęć piłki.

Klasa: **Trener**

- Klasa reprezentująca trenera drużyny lub klubu sportowego.
 1. dataZatrudnienia : Date – data rozpoczęcia pracy trenera w klubie.
 2. klub : Klub – klub, w którym trener jest zatrudniony.
 3. Klub() : Klub – zwraca klub, do którego przypisany jest trener.

Klasa: **Uzytkownik**

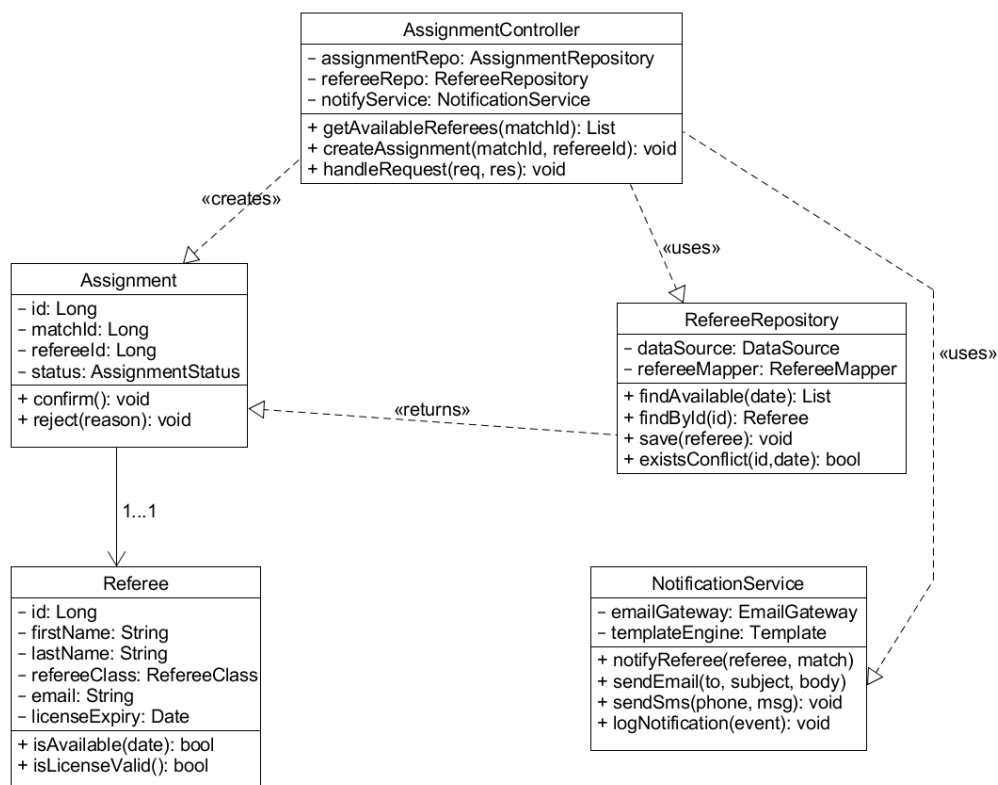
- Klasa reprezentująca użytkownika systemu, odpowiedzialna za przechowywanie danych logowania oraz informacji o uprawnieniach.
 1. id : Int – unikalny identyfikator użytkownika.
 2. login : String – nazwa użytkownika wykorzystywana podczas logowania.
 3. rola : String – rola użytkownika w systemie, określająca poziom uprawnień (np. administrator, trener, zawodnik).
 4. autoryzuj(Uzytkownik) – weryfikuje dane użytkownika i sprawdza, czy posiada on uprawnienia do uzyskania dostępu do systemu lub określonych funkcji.

Klasa: **Zawodnik**

- Klasa reprezentująca zawodnika drużyny wraz z jego danymi osobowymi oraz statystykami meczowymi.
 1. numer : int – numer zawodnika na koszulce.
 2. imię : String – imię zawodnika.
 3. nazwisko : String – nazwisko zawodnika.
 4. pozycja : String – pozycja zawodnika na boisku.
 5. kapitan : bool – informacja określająca, czy zawodnik jest kapitanem drużyny.
 6. Pts : int – liczba zdobytych punktów.
 7. Att : int – liczba wykonanych ataków.
 8. Succ : int – liczba skutecznych akcji lub udanych ataków.
 9. Aces : int – liczba asów serwisowych.
 10. Blk : int – liczba bloków.
 11. Digs : int – liczba obronionych piłek (wykopów/interwencji defensywnych).

Klasa: **Drużyna**

- Klasa zawierająca informacje o drużynie
4. Członkowie : `Array[Zawodnik]` – tablica zawierająca referencję do klasy Zawodnik.
 5. `średnia_liczba_punktów` : `int` – średnia punktów uzyskujących ze wszystkich meczy rozegranych w ogóle.
 6. `Kapitan()`: Zawodnik – metoda zwraca kapitana drużyny
 7. `ProcentZwycęstw()` : `float` – zwraca procent wygranych meczy
 8. `Rekord()` : `Tuple<int,int>` – zwraca parę liczb <rekord największej liczby punktów zdobytych podczas tego meczu, ID meczu>



Rysunek 10 Diagram Klas do przypadku użycia “Przydzielenia sędziego do meczu”

Klasa: **AssignmentController**

- Klasa odpowiedzialna za obsługę procesu przydzielania sędziego do meczu. Pobiera dane z repozytoriów i wywołuje powiadomienia.
1. `assignmentRepo: AssignmentRepository` – dostęp do danych dotyczących przydziałów sędziów.
 2. `refereeRepo: RefereeRepository` – dostęp do danych sędziów
 3. `notifyService: NotificationService` – serwis wysyłający powiadomienia.
 4. `getAvailableReferees(matchId): List` – zwraca listę dostępnych sędziów dla wybranego meczu.

5. createAssignment(matchId, refereeId): void – tworzy nowy przydział sędziego do meczu.
6. handleRequest(req, res): void – obsługuje żądania użytkownika związane z przydzielaniem sędziów.

Klasa: **Assignment**

- Klasa reprezentująca przydział sędziego do konkretnego meczu.
 1. id: Long – unikalny identyfikator przydziału
 2. matchId: Long – identyfikator meczu.
 3. refereeId: Long – identyfikator przypisanego sędziego.
 4. status: AssignmentStatus – aktualny status przydziału.
 5. confirm(): void – potwierdza przyjęcie przydziału.
 6. reject(reason): void – odrzuca przydział wraz z podaniem powodu.

Klasa: **Referee**

- Klasa reprezentująca sędziego sportowego.
 1. id: Long – identyfikator sędziego.
 2. firstName: String – imię sędziego.
 3. lastName: String – nazwisko sędziego.
 4. refereeClass: RefereeClass – klasa/uprawnienia sędziego.
 5. email: String – adres e-mail sędziego.
 6. licenseExpiry: Date – data ważności licencji.
 7. isAvailable(date): bool – sprawdza dostępność sędziego w danym terminie.
 8. isLicenseValid(): bool – sprawdza ważność licencji sędziego.

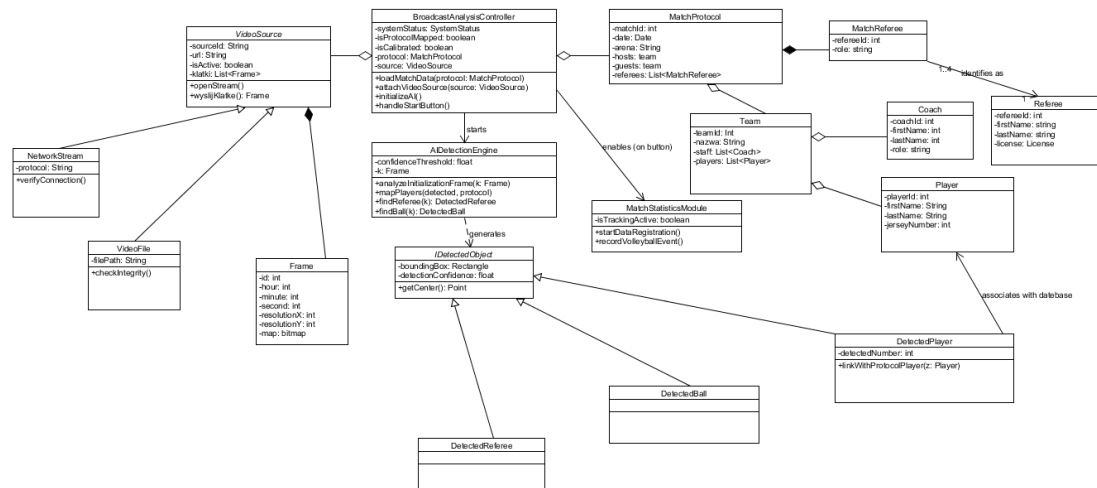
Klasa: **RefereeRepository**

- Klasa odpowiedzialna za komunikację z bazą danych dotyczącą sędziów.
 1. dataSource: DataSource – źródło połączenia z bazą danych.
 2. refereeMapper: RefereeMapper – mapuje dane z bazy na obiekty Referee.
 3. findAvailable(date): List – wyszukuje dostępnych sędziów w danym terminie.
 4. findById(id): Referee – pobiera sędziego na podstawie identyfikatora.
 5. save(referee): void – zapisuje dane sędziego do bazy
 6. existsConflict(id, date): bool – sprawdza konflikt terminów dla sędziego.

Klasa: **NotificationService**

- Klasa odpowiedzialna za wysyłanie powiadomień do użytkowników systemu.
 1. emailGateway: EmailGateway – komponent do wysyłania wiadomości e-mail.
 2. templateEngine: Template – mechanizm generowania szablonów wiadomości.
 3. notifyReferee(referee, match) – wysyła sędziemu informację o przydziale do meczu.
 4. sendEmail(to, subject, body) – wysyła wiadomość e-mail.
 5. sendSms(phone, msg): void – wysyła wiadomość SMS.
 6. logNotification(event): void – zapisuje historię wysłanych powiadomień.

Karol Kuc



Rysunek 11 Diagram klas dla zdarzenia „Przygotuj analizę transmisji wideo”

1. AIDetectionEngine

- **Opis klasy:** Klasa odpowiedzialna za warstwę logiczną sztucznej inteligencji i Computer Vision. Analizuje surowe klatki obrazu w celu wykrywania obiektów na boisku i mapowania ich na rzeczywiste encje biznesowe.
- **Atrybuty:**
 - confidenceThreshold: float – Próg pewności algorytmu, określający minimalne prawdopodobieństwo (np. 0.75), powyżej którego detekcja obiektu uznawana jest za poprawną.
- **Metody:**
 - analyzeInitializationFrame(k: Frame) – Analizuje pierwszą/inicjalizacyjną klatkę wideo w celu kalibracji przestrzeni boiska i wykrycia stałych elementów.
 - mapPlayers(detected, protocol) – Odpowiada za algorytm dopasowania (asocjacji) fizycznie wykrytych na nagraniu postaci do zawodników wpisanych w oficjalnym protokole meczowym.
 - findReferee(k: Frame): DetectedReferee – Przetwarza klatkę obrazu za pomocą sieci neuronowej w celu zlokalizowania pozycji sędziego.
 - findBall(k: Frame): DetectedBall – Śledzi i lokalizuje pozycję piłki siatkarskiej na podanej klatce wideo.

2. BroadcastAnalysisController

- **Opis klasy:** Główny kontroler biznesowy, który zarządza całym procesem przygotowania i uruchomienia analizy transmisji meczowej. Spaja warstwę danych wejściowych z silnikiem AI i modułem statystyk. Z uwagi na charakterystykę narzędzia UMLet, powiązania z MatchProtocol oraz VideoSource zostały zaimplementowane za pomocą notacji agregacji (pusty romb), choć logicznie reprezentują zależność cyklu życia w ramach tego scenariusza.
- **Atrybuty:**
 - systemStatus: SystemStatus – Reprezentuje aktualny stan maszyny stanowej systemu (np. Oczekiwanie, Kalibracja, Analiza).
 - isProtocolMapped: boolean – Flaga informująca, czy lista zawodników z wideo została pomyślnie powiązana z bazą danych protokołu.
 - isCalibrated: boolean – Flaga określająca, czy system wizyjny pomyślnie przeszedł kalibrację kamer i geometrii boiska.
 - protocol: MatchProtocol – Referencja do aktualnie ładowanego protokołu meczowego.
 - source: VideoSource – Referencja do podpętego źródła strumienia lub pliku wideo.
- **Metody:**
 - loadMatchData(protocol: MatchProtocol) – Wczytuje dane meczu z bazy danych na podstawie wybranego protokołu.
 - attachVideoSource(source: VideoSource) – Inicjalizuje połączenie ze wskazanym plikiem wideo lub sieciowym strumieniem RTSP/RTMP.
 - initializeAI() – Uruchamia procesy ładowania modeli wagowych sieci neuronowych do pamięci graficznej (GPU).
 - handleStartButton() – Metoda obsługująca zdarzenie kliknięcia przycisku "Start" przez użytkownika, rozpoczynająca właściwe przetwarzanie i zbieranie statystyk.

3. DetectedBall

- **Opis klasy:** Klasa implementująca interfejs IDetectedObject. Reprezentuje instancję piłki wykrytej w konkretnym punkcie przestrzeni i czasie przez model AI. Dziedziczy strukturę i zachowanie po klasie bazowej.
- **Atrybuty:** (Dziedziczy strukturę po IDetectedObject)
- **Metody:** (Dziedziczy zachowanie po IDetectedObject)

4. DetectedPlayer (WykrytyZawodnik)

- **Opis klasy:** Klasa implementująca interfejs IDetectedObject. Reprezentuje wysegmentowaną postać zawodnika na nagraniu wideo, wzbogaconą o odczytany przez algorytm OCR numer na koszulce. Dziedziczy strukturę po klasie bazowej
- **Atrybuty:** (Dziedziczy strukturę po IDetectedObject)
 - detectedNumber: int – Numer na koszulce zawodnika rozpoznany automatycznie przez algorytmy optycznego rozpoznawania znaków.
- **Metody:** (Dziedziczy zachowanie po IDetectedObject)
 - linkWithProtocolPlayer(z: Player) – Tworzy trwałe logiczne powiązanie pomiędzy wykrytym obiektem wizyjnym a konkretnym zawodnikiem z bazy danych.

5. DetectedReferee

- **Opis klasy:** Klasa implementująca interfejs IDetectedObject. Reprezentuje postać sędziego (głównego, pomocniczego lub liniowego) wykrytą i śledzoną w kadrze kamery. Dziedziczy strukturę po klasie bazowej.
- **Atrybuty:** (Dziedziczy strukturę po IDetectedObject)
- **Metody:** (Dziedziczy zachowanie po IDetectedObject)

6. Frame

- **Opis klasy:** Klasa strukturalna reprezentująca pojedynczą ramkę obrazu (klatkę wideo) wyciągniętą ze źródła wideo, przekazywaną bezpośrednio do analizy przez AI.
- **Atrybuty:**
 - id: int – Unikalny numer sekwencyjny klatki w strumieniu wideo.
 - hour: int / minute: int / second: int – Znacznik czasowy (timestamp) rejestracji klatki.
 - resolutionX: int / resolutionY: int – Szerokość i wysokość obrazu wyrażona w pikselach.
 - map: bitmap – Surowa macierz pikseli (bitmapa) przechowywana w pamięci podręcznej.

7. IDetectedObject

- **Opis klasy:** Interfejs stanowiący wspólny kontrakt dla wszystkich obiektów fizycznych, które system wizyjny potrafi zidentyfikować na boisku.
- **Atrybuty:**

- `boundingBox`: `Rectangle` – Prostokąt otaczający obiekt, definiujący jego współrzędne (X, Y, szerokość, wysokość) na ekranie.
- `detectionConfidence`: `float` – Współczynnik pewności (od 0 do 1) dla tej konkretnej, pojedynczej detekcji obiektu.
- **Metody:**
 - `getCenter()`: `Point` – Oblicza i zwraca punkt centralny (środek ciężkości geometrii) wykrytego prostokąta, kluczowy dla śledzenia trajektorii ruchu.

8. MatchProtocol

- **Opis klasy:** Klasa reprezentująca dokumentację i komplet formalnych danych o meczu pobranych z bazy danych przed rozpoczęciem analizy. Stanowi logiczną całość agregującą obiekty drużyn oraz sędziów.
- **Atrybuty:**
 - `matchId`: `int` – Unikalny identyfikator spotkania w bazie danych.
 - `date`: `Date` – Data i godzina rozegrania meczu.
 - `arena`: `String` – Nazwa hali sportowej / obiektu, na którym odbywa się spotkanie.
 - `hosts`: `team` / `guests`: `team` – Wskazanie drużyny gospodarzy oraz drużyny gości.
 - `referees`: `List<MatchReferee>` – Oficjalna lista sędziów wyznaczonych do obsługi tego spotkania.

9. MatchReferee

- **Opis klasy:** Klasa pośrednicząca, która przypisuje konkretnego sędziego z bazy danych do danego protokołu meczowego wraz z określeniem jego aktualnej funkcji i roli.
- **Atrybuty:**
 - `refereeId`: `int` – Klucz obcy wskazujący na sędziego.
 - `role`: `string` – Specyficzna rola sędziego w tym konkretnym meczu (np. sędzia główny, sędzia pierwszy, sędzia drugi, sędzia liniowy).

10. MatchStatisticsModule

- **Opis klasy:** Moduł odpowiedzialny za przetwarzanie poprawnie zmapowanych danych przestrzennych z AI na finalne statystyki sportowe i zdarzenia boiskowe.
- **Atrybuty:**
 - `isTrackingActive`: `boolean` – Stan informujący, czy moduł aktualnie rejestruje i zapisuje zdarzenia w locie.
- **Metody:**

- startDataRegistration() – Uruchamia proces nasłuchiwania danych z silnika AI i otwierania sesji zapisu statystyk.
- recordVolleyballEvent() – Analizuje i zapisuje w bazie zidentyfikowane zdarzenie siatkarskie (np. atak, blok, zagrywka, as serwisowy).

11. NetworkStream

- **Opis klasy:** Specjalizacja źródła wideo reprezentująca bezpośredni strumień cyfrowy przesyłany po sieci (np. z kamery zamontowanej na hali). Dziedziczy po VideoSource.
- **Atrybuty:** (Dziedziczy strukturę po VideoSource)
 - protocol: String – Protokół sieciowy wykorzystywany do transmisji (np. "RTSP", "RTMP", "HLS").
- **Metody:** (Dziedziczy zachowanie po VideoSource)
 - verifyConnection() – Sprawdza stabilność połączenia sieciowego z kamerą, opóźnienia (ping) oraz dostępność strumienia pod podanym adresem URL.

12. Player

- **Opis klasy:** Klasa encyjna przechowująca stałe dane zawodnika zapisane w strukturze klubu/drużyny. Wchodzi w skład relacji agregacji wewnątrz klasy Team.
- **Atrybuty:**
 - playerId: int – Unikalny numer identyfikacyjny zawodnika w systemie.
 - firstName: String / lastName: String – Imię i nazwisko sportowca.
 - jerseyNumber: int – Oficjalny przypisany numer na koszulce zawodnika w kadrze.

13. Coach

- **Opis klasy:** Klasa reprezentująca dane personalne oraz rolę trenera przypisanego do danego sztabu szkoleniowego drużyny. Związana relacją agregacji wewnątrz struktury zespołu.
- **Atrybuty:**
 - coachId: int – Unikalny numer identyfikacyjny trenera w systemie.
 - firstName: String / lastName: String – Imię i nazwisko trenera.

- role: String: - Rola trenera w drużynie, (pierwszy trener, drugi trener, trener przygotowania fizycznego itp.)

14. Referee

- **Opis klasy:** Klasa encyjna przechowująca globalne dane osobowe sędziego zarejestrowanego w systemie ligowym. Połączona z protokołem za pomocą klasy asocjacyjnej.
- **Atrybuty:**
 - refereeId: int – Unikalny identyfikator sędziego.
 - firstName: string / lastName: string – Imię i nazwisko sędziego.
 - license: License – Poziom lub numer posiadanej licencji sędziowskiej uprawniającej do prowadzenia meczów określonej rangi.

15. Team

- **Opis klasy:** Klasa encyjna reprezentująca klub lub zespół siatkarski, zawierająca pełną strukturę sztabu i powołanych zawodników. Na diagramie występuje jako składowa agregacji wewnątrz protokołu meczowego.
- **Atrybuty:**
 - teamId: Int – Unikalny identyfikator zespołu.
 - firstName: String / lastName: String – Pola przeznaczone na identyfikację nazwy i skrótu reprezentatywnego klubu.
 - staff: List<Coach> – Lista członków sztabu szkoleniowego przypisana do drużyny.
 - players: List<Player> – Kadra zawodnicza wchodząca w skład danej drużyny.

16. VideoFile

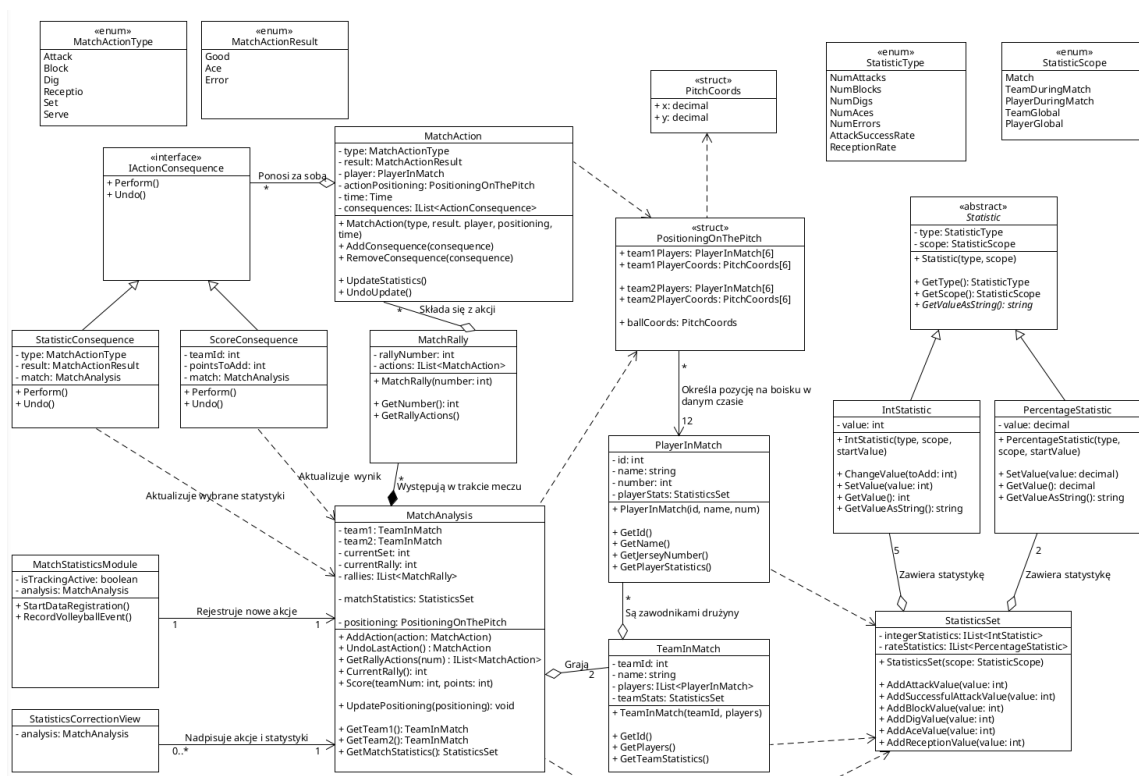
- **Opis klasy:** Specjalizacja źródła wideo reprezentująca statyczne, uprzednio nagrane i zapisane na dysku lokalnym nagranie meczu (np. w formacie .mp4). Dziedziczy po VideoSource
- **Atrybuty:** (Dziedziczy strukturę po VideoSource)
 - filePath: String – Ścieżka dostępu do pliku w systemie operacyjnym lub pamięci masowej.
- **Metody:** (Dziedziczy zachowanie po VideoSource)

- `checkIntegrity()` – Sprawdza poprawność pliku wideo, weryfikuje kodeki oraz nagłówki pliku, upewniając się, że plik nie jest uszkodzony.

17. VideoSource

- **Opis klasy:** Klasa abstrakcyjna definiująca ogólny interfejs dla dostarczania obrazu wideo do aplikacji, niezależnie od tego, czy obraz pochodzi z pliku, czy z kamery sieciowej na żywo.
- **Atrybuty:**
 - `sourceId`: String – Unikalny identyfikator sprzętowy lub systemowy źródła.
 - `url`: String – Adres sieciowy lub ścieżka zasobu źródłowego.
 - `isActive`: boolean – Informacja o tym, czy źródło jest w danym momencie otwarte i nadaje obraz.
- **Metody:**
 - `openStream()` – Rezerwuje zasoby systemowe i otwiera kanał odczytu wideo.
 - `fetchFrame()`: Frame – Wyciąga pojedynczą, aktualną klatkę obrazu ze strumienia dekodera i zwraca ją w postaci obiektu klasy Frame.

Dominik Jarzyło



Rysunek 12 Diagram klas dla przypadku użycia „Obserwuj analizę dokonywaną przez AI na żywo”

1. IActionConsequence

- **Opis klasy:** Interfejs reprezentujący pojedynczą konsekwencję wybranej akcji. Może nią być np. zwiększenie wartości danej statystyki lub zdobycie punktów przez drużynę.
- **Metody:**
 - Perform() – Wykonuje właściwą operację związaną z konsekwencją, np. zwiększenie wartości statystyki,
 - Undo() – cofa wykonanie operacji związanej z konsekwencją.

2. IntStattitic

- **Opis klasy:** Specjalizacja klasy **Statistic**; reprezentuje statystykę, którą da się zapisać jako liczbę całkowitą (int).
- **Atrybuty:** (Dziedziczy strukturę po **Statistic**)
 - value – przechowuje obecną wartość statystyki.
- **Metody:** (Dziedziczy zachowanie po **Statistic**)
 - IntStatistic(type, scope, startValue) – konstruktor,
 - ChangeValue(toAdd: int) – zmienia statystykę o podaną wartość
 - SetValue(value: int)
 - GetValue(): int
 - GetValueAsString(): string – zwraca wartość statystyki w postaci tekstowej.

3. PercentageStatistic

- **Opis klasy:** Specjalizacja klasy **Statistic**; reprezentuje statystykę, która jest zapisywana w postaci procentowej, np. procent skutecznych ataków.
- **Atrybuty:** (Dziedziczy strukturę po **Statistic**)
 - value – przechowuje obecną wartość statystyki.
- **Metody:** (Dziedziczy zachowanie po **Statistic**)
 - PercentageStatistic(type, scope, startValue) – konstruktor,
 - SetValue(value: decimal)
 - GetValue(): decimal
 - GetValueAsString() – zwraca wartość statystyki w postaci tekstowej.

4. PitchCoords

- **Opis klasy:** Struktura określająca pozycję (koordynaty) na boisku.
- **Atrybuty:**
 - x – pozycja wzdłuż boiska,
 - y – pozycja wszerz boiska.

5. PlayerInMatch

- **Opis klasy:** Klasa reprezentuje zawodnika w trakcie trwania meczu oraz przechowuje podstawowe informacje o nim i jego meczowe statystyki.
- **Atrybuty:**
 - id: int – ID zawodnika,
 - name: string – nazwisko zawodnika,
 - number – numer zawodnika na koszulce,
 - playerStats – statystyki zawodnika w trakcie meczu.
- **Metody:**
 - PlayerInMatch(id, name, num) – konstruktor
 - GetId()
 - GetName()
 - GetJerseyNumber()
 - GetPlayerStatistics()

6. PositioningOnThePitch

- **Opis klasy:** Struktura reprezentująca rotacje i pozycje graczy obu drużyn na boisku w danym czasie oraz lokalizację piłki.
- **Atrybuty:**
 - team1Players: PlayerInMatch[6] – obecnie wystawieni zawodnicy należący do pierwszej drużyny,
 - team1PlayerCoords: PitchCoords[6] – dokładne koordynaty zawodników należących do pierwszej drużyny,
 - team2Players: PlayerInMatch[6] – obecnie wystawieni zawodnicy należący do drugiej drużyny,
 - team2PlayerCoords: PitchCoords[6] – dokładne koordynaty zawodników należących do drugiej drużyny.

7. MatchAction

- **Opis klasy:** Klasa reprezentująca pojedynczą akcję wykonywaną przez zawodnika w trakcie meczu podczas wymiany (Rally).
- **Atrybuty:**
 - Type: MatchActionType – rodzaj akcji,
 - result: MatchActionResult – rezultat akcji; określa, czy była ona wykonana poprawnie, czy skutkowałą zdobyciem punktu (as), czy była błędem,
 - player: PlayerInMatch – gracz wykonujący akcję,
 - actionPositioning: PositioningOnThePitch – ustawienie zawodników i lokalizacja piłki w momencie wykonania akcji
 - time: Time – określa czas w meczu, w którym została wykonana akcja,
 - consequences: IList<IActionConsequence> – lista konsekwencji, czyli zmian w statystykach lub punktacji po wykonaniu akcji.
- **Metody:**
 - MatchAction(type, result, player, positioning, time) – konstruktor,
 - AddConsequence(consequence) – dodaje konsekwencję akcji,
 - RemoveConsequence(consequence) – usuwa wcześniej dodaną konsekwencję akcji, np. podczas dokonywania korekty.
 - UpdateStatistics() – aktualizuje statystyki i punktacje po wykonaniu akcji poprzez wywołanie metod Perform() powiązanych obiektów konsekwencji,
 - UndoUpdate() – cofa zmiany w statystykach i punktacji wykonane wcześniej przez metodę UpdateStatistics() poprzez wywołanie metod Undo() powiązanych obiektów konsekwencji.

8. MatchActionType

- **Opis klasy:** Typ wyliczeniowy określający rodzaj akcji.
- **Wartości:**
 - Attack
 - Block
 - Dig
 - Reception
 - Set
 - Serve

9. MatchActionResult

- **Opis klasy:** Typ wyliczeniowy określający rezultat akcji.
- **Wartości:**
 - Good – prawidłowo wykonana zagrywka niekończąca się uzyskaniem punktu przez którąkolwiek z drużyn,
 - Ace – zagrywka zakończyła się pomyślnym zdobyciem punktu,
 - Error – zagrywka poskutkowałą zdobyciem punktu przez drużynę przeciwną.

10. MatchAnalysis

- **Opis klasy:** Klasa przechowująca informacje o całości dokonywanej analizy, jej obecny stan, statystyki, oraz historię wszystkich akcji.
- **Atrybuty:**
 - team1: TeamInMatch – pierwsza drużyna biorąca udział w meczu,
 - team2: TeamInMatch – druga drużyna biorąca udział w meczu,
 - currentSet: int – obecny set,
 - currentRally: int – numer obecnej wymiany (Rally),
 - rallies: IList<MatchRally> – lista wszystkich wykonywanych do tej pory wymian,
 - matchStatistics: StatisticsSet – statystyki dotyczące ogółu meczu,
 - positioning – obecne ustawienie zawodników i lokalizacja piłki na boisku.
- **Metody:**
 - AddAction(action: MatchAction) – dodaje akcję do obecnej wymiany i aktualizuje statystyki,
 - UndoLastAction() : MatchAction – cofa ostatnią zarejestrowaną akcję i zwraca jej obiekt,
 - GetRallyActions(numRally: int): IList<MatchRally> – zwraca listę akcji należących do wymiany o podanym numerze,
 - CurrentRally(): int – zwraca numer obecnej wymiany,
 - Score(teamNum: int, points: int) – zmienia liczbę punktów danej drużyny o podaną wartość,
 - UpdatePositioning(positioning) – aktualizuje obecne ustawienie na boisku.
 - GetTeam1(): TeamInMatch
 - GetTeam2(): TeamInMatch
 - GetMatchStatistics(): StatisticsSet

11. MatchRally

- **Opis klasy:** Klasa reprezentująca pojedynczą wymianę w trakcie meczu.
- **Atrybuty:**
 - rallyNumber: int – numer wymiany,
 - actions: IList<MatchAction> – akcje należące do wymiany.
- **Metody:**
 - MatchRally(number: int) – konstruktor.
 - GetNumber(): int
 - GetRallyActions: IList<MatchAction>

12. MatchStatisticsModule

- **Opis klasy:** Moduł tłumaczący zmapowane przez AI dane przestrzenne na zdarzenia na boisku. Dodaje akcje wraz z ich konsekwencjami do analizy meczu (MatchAnalysis).
- **Atrybuty:**
 - IsTrackingActive
 - analysis – referencja na obiekt przechowujący dane analizy.
- **Metody:**
 - StartDataRegistration()
 - RecordVolleyballEvent()

13. ScoreConsequence

- **Opis klasy:** Implementacja interfejsu IActionConsequence. Klasa reprezentuje konsekwencję akcji będącą uzyskaniem punktów przez jedną z drużyn.
- **Atrybuty:**
 - teamId – ID drużyny, która ma zdobyć punkty,
 - pointsToAdd: int – liczba uzyskanych punktów,
 - match: MatchAnalysis – referencja na obiekt analizy meczu.
- **Metody:**
 - Perform() – aktualizuje liczbę punktów drużyny w analizie meczu,
 - Undo() – przywraca poprzednią liczbę punktów drużyny.

14. Statistic

- **Opis klasy:** Klasa abstrakcyjna reprezentująca pewną statystykę.
- **Atrybuty:**
 - type: StatisticType – rodzaj statystyki,
 - scope: StatisticScope – zakres statystyki.
- **Metody:**
 - Statistic(type, scope) – konstruktor.
 - GetType(): StatisticType
 - GetScope(): StatisticScope
 - GetValueAsString(): string – zwraca wartość statystyki w postaci tekstowej.

15. StatisticConsequence

- **Opis klasy:** Implementacja interfejsu IActionConsequence. Klasa reprezentuje konsekwencję zdarzenia na boisku w postaci zmiany wartości pewnych statystyk. Wartości i rodzaj statystyk, jakie mają zostać zaktualizowane wyznaczone są na podstawie rodzaju i rezultatu akcji.
- **Atrybuty:**
 - type: MatchActionType
 - result: MatchActionResult
 - match: MatchAnalysis – referencja na obiekt analizy meczu.
- **Metody:**
 - Perform() – aktualizuje odpowiednie statystyki,
 - Undo() – przywraca poprzednie wartości statystyk.

16. StatisticScope

- **Opis klasy:** Typ wyliczeniowy określający zakres danej statystyki.
- **Wartości:**
 - Match – statystyka dotyczy ogółu meczu,
 - TeamDuringMatch – statystyka dotyczy gry drużyny w trakcie jednego meczu,
 - PlayerDuringMatch – statystyka dotyczy gry pojedynczego zawodnika w trakcie jednego meczu,
 - TeamGlobal – statystyka dotyczy gry drużyny ogółem we wszystkich zarejestrowanych meczach,
 - PlayerGlobal – statystyka dotyczy gry pojedynczego zawodnika ogółem we wszystkich zarejestrowanych meczach,

17. StatisticsCorrectionView

- **Opis klasy:** Klasa reprezentująca widok służący do korekcji danych przez statystyka
- **Atrybuty:**
 - analysis: MatchAnalysis – referencja na obiekt analizy meczu.

18. StatisticsSet

- **Opis klasy:** Zestaw statystyk o danym zakresie.
- **Atrybuty:**
 - integerStatistics: IList<IntStatistic> – lista statystyk o wartościach całkowitych,
 - rateStatistics: IList<PercentageStatistic> – lista statystyk procentowych. Aktualizowane automatycznie.
- **Metody:**
 - StatisticsSet(scope: StatisticScope) – konstruktor.
 - AddAttackValue(value: int)
 - AddSuccessfulAttackValue(value: int)
 - AddBlockValue(value: int)
 - AddDigValue(value: int)
 - AddAceValue(value: int)
 - AddReceptionValue(value: int)

19. StatisticType

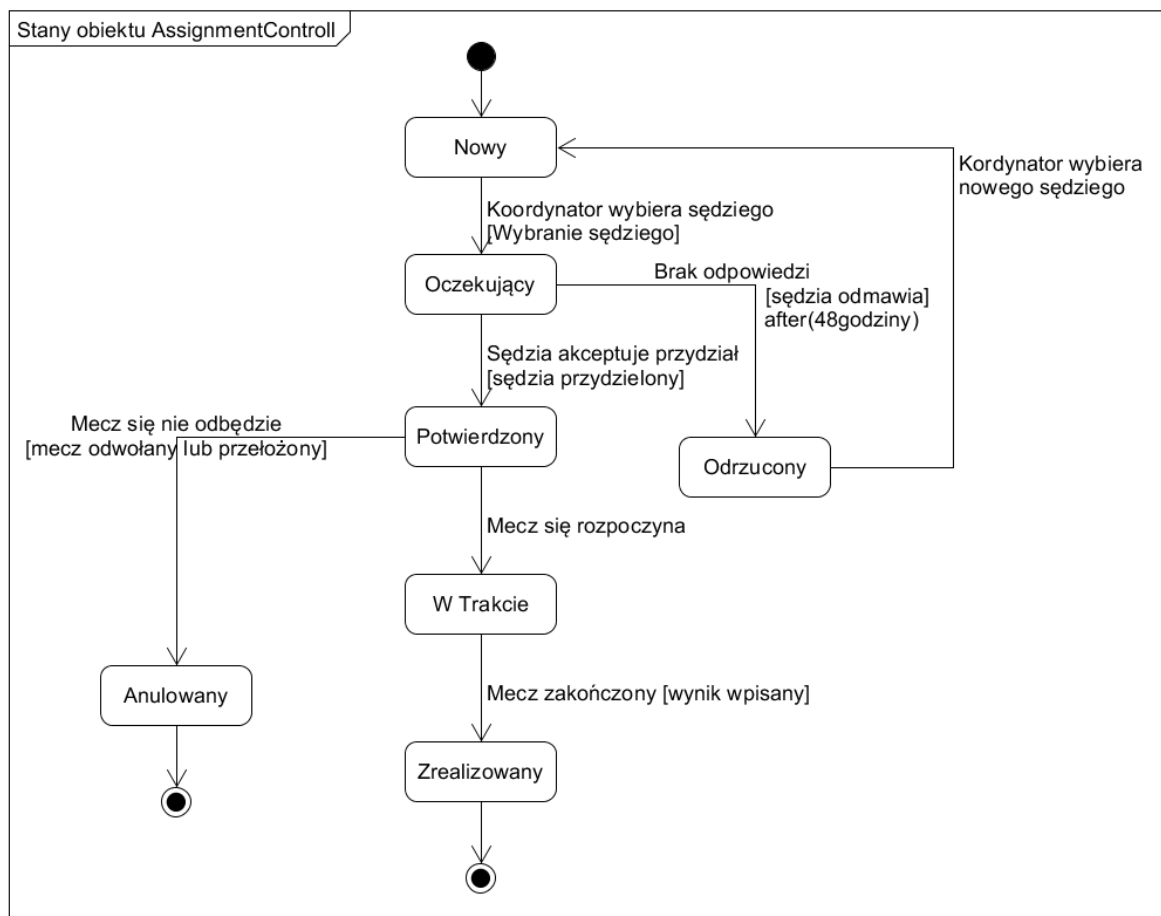
- **Opis klasy:** Typ wyliczeniowy określający rodzaj statystyki.
- **Wartości:**
 - NumAttacks
 - NumBlocks
 - NumDigs
 - NumAces
 - NumErrors
 - AttackSuccessRate
 - ReceptionRate

20. TeamInMatch

- **Opis klasy:** Reprezentuje drużynę, jej skład i statystyki w trakcie meczu.
- **Atrybuty:**
 - teamId: int – ID drużyny,
 - name: string – nazwa drużyny,
 - players: IList<PlayerInMatch> – lista zawodników drużyny biorących udział w danym meczu.
- **Metody:**
 - TeamInMatch(teamId, players) – konstruktor.
 - GetId()
 - GetPlayers()
 - GetTeamStatistics()

Diagramy stanów

Mateusz Tręda



Rysunek 13 Diagram stanów dla obiektu AssignmentControll

Stan: NOWY

stan początkowy tworzony automatycznie przy planowaniu meczu. Brak jeszcze przydzielonego sędziego. System wstępnie pobiera listę dostępnych kandydatów. Przejście wychodzące: koordynator wybiera sędziego → OCZEKUJĄCY.

Stan: OCZEKUJĄCY

przydział zaproponowany, system wysyła powiadomienie e-mail/SMS do sędziego. Uruchamia się licznik 48 godzin. Dwa możliwe przejścia: sędzia akceptuje → POTWIERDZONY; sędzia odmawia lub mija timeout → ODRZUCONY.

Stan: POTWIERDZONY

sędzia potwierdził gotowość, obsada jest kompletna. System wysyła potwierdzenie koordynatorowi. Możliwe przejścia: mecz się rozpoczyna → W_TRAKCIE; mecz zostaje odwołany/przełożony → ANULOWANY

Stan: W_TRAKCIE

mecz aktualnie trwa, sędzia jest na boisku. System rejestruje czas startu. Jedyne przejście wychodzące: mecz kończy się i wynik jest wpisany → ZREALIZOWANY.

Stan: ZREALIZOWANY

stan końcowy powodzenia. Przydział zakończony. System aktualizuje historię i statystyki sędziego. Brak przejść wychodzących.

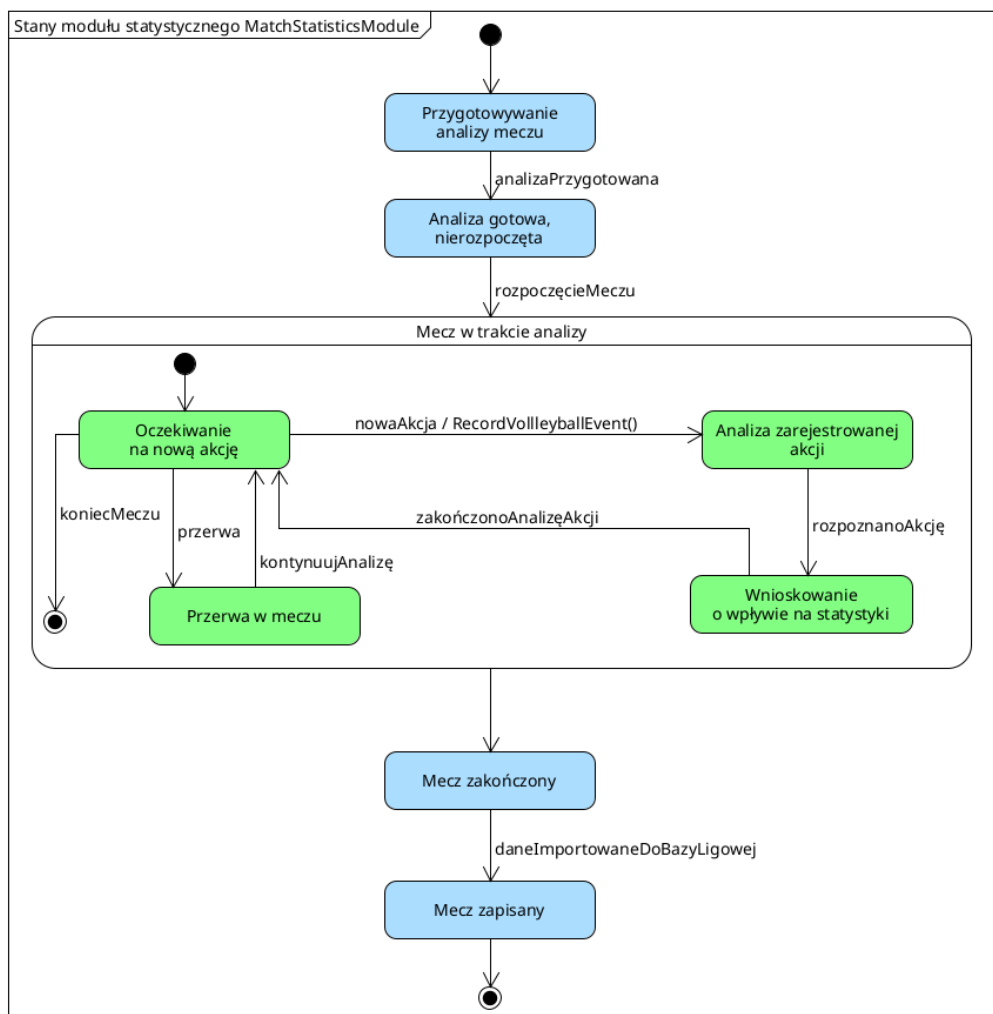
Stan: ODRZUCONY

sędzia odmówił lub nie odpowiedział w ciągu 48h. System generuje alert dla koordynatora. Nie jest to stan końcowy — koordynator wraca do stanu NOWY (linia przerywana), aby wybrać innego sędziego.

Stan: ANULOWANY

stan końcowy awarii organizacyjnej. Mecz odwołany lub przełożony decyzją PZPS. System zwalnia sędziego i anuluje wszystkie powiązane rezerwacje.

Dominik Jarzyło



Rysunek 14 Diagram stanów dla klasy MatchStatisticsModule

Stan: Przygotowywanie analizy meczu

Stan początkowy, przyjmowany przez obiekt klasy MatchStatisticsModule po utworzeniu. Nie są jeszcze zarejestrowane źródła, z których będzie pobierane nagranie do analizy.

Stan: Analiza gotowa, nierozpoczęta

Stan przybierany po zarejestrowaniu źródeł nagrań, na podstawie których będzie dokonywana analiza meczu. Obiekt klasy czeka na rozpoczęcie meczu.

Stan: Mecz w trakcie analizy

Stan złożony, w jakim obiekt klasy MatchStatisticsModule znajduje się w trakcie meczu. Prowadzona jest analiza akcji i statystyk.

Stan: Oczekiwanie na nową akcję

Podstan stanu „Mecz w trakcie analizy”. W tym stanie aktywnie monitorowana jest aktywność zawodników na boisku. System oczekuje na wykonanie akcji przez dowolnego gracza, w której wyniku rejestrowane jest nowe zdarzenie boiskowe i moduł statystyczny przechodzi w stan dogłębnej analizy wykonanej przez siatkarza akcji. Wyjście ze stanu może nastąpić również w wyniku zakończenia meczu lub rozpoczęcia przerwy.

Stan: Analiza zarejestrowanej akcji

Podstan stanu „Mecz w trakcie analizy”. System dokładnie analizuje działania zawodnika na podstawie nagrań. Określany jest gracz wykonujący akcję, typ akcji i jej rezultat (np. akcja została wykonana skutecznie lub była błędna).

Stan: Wnioskowanie o wpływie na statystyki

Podstan stanu „Mecz w trakcie analizy”. Na podstawie danych zebranych z analizy AI nagrań z kamer następuje wnioskowanie o wpływie ostatnio zarejestrowanej akcji na statystyki meczu. Określane są konsekwencje akcji, system decyduje o ewentualnym przyznaniu punktów jednej z drużyn i o konieczności zmiany rotacji. Po zakończeniu wnioskowania dane zapisywane są przez obiekt klasy MatchAnalysis i następuje powrót do stanu „Oczekiwanie na nową akcję”.

Stan: Przerwa w meczu

Podstan stanu „Mecz w trakcie analizy”. W tym stanie analiza zdarzeń na boisku jest wstrzymana do momentu zakończenia przerwy.

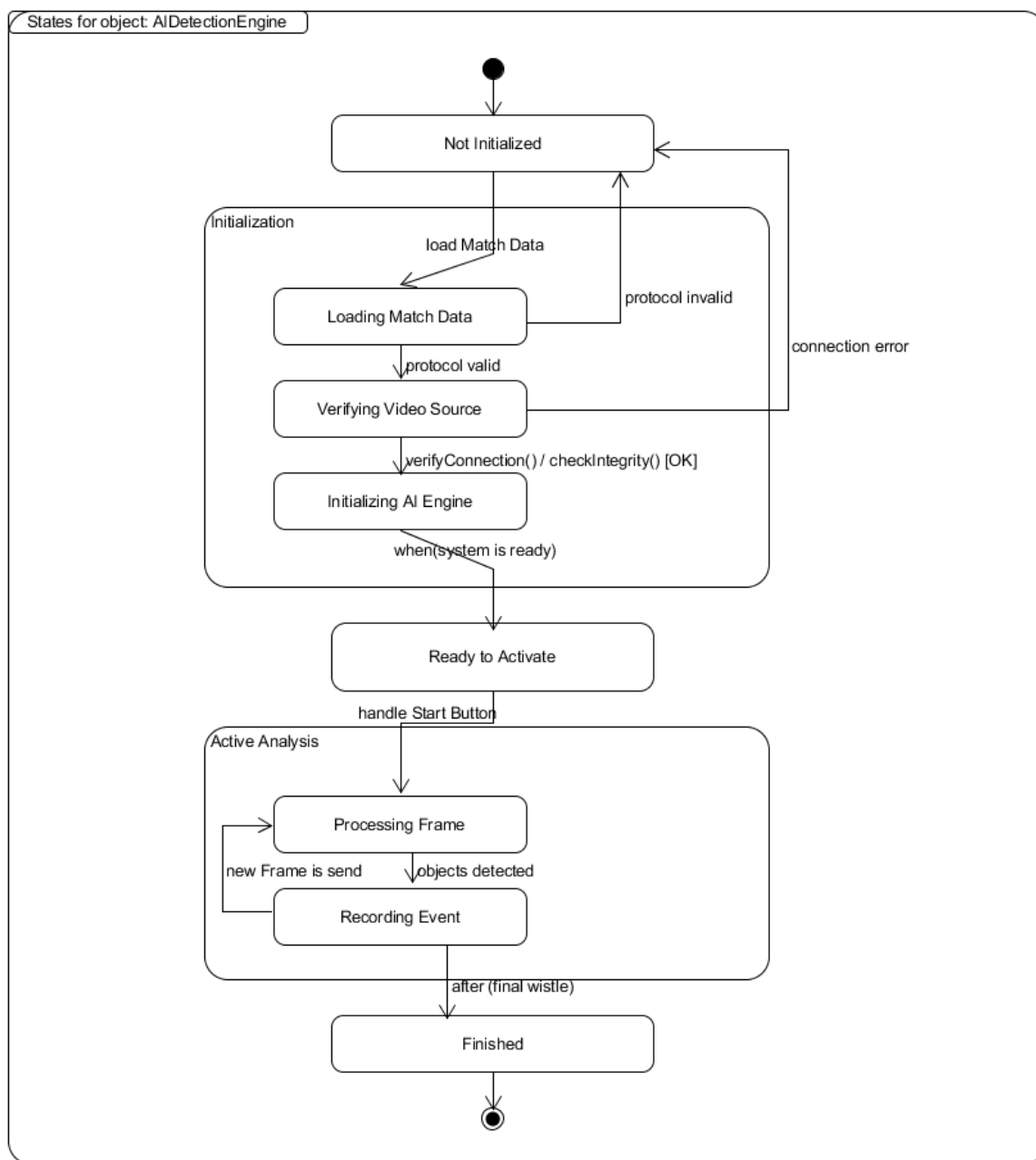
Stan: Mecz zakończony

Automatyczna analiza meczu została zakończona. Jednak dane nie zostały jeszcze zapisane do bazy danych – wciąż możliwa jest ich ewentualna weryfikacja i dokonanie korekty przez statystyka.

Stan: Mecz zapisany

Stan końcowy. Analiza meczu została zakończona, a dane wynikowe zostały zapisane w ligowej bazie danych.

Karol Kuc



Rysunek 15 Diagram stanu dla obiektu AiDetectionEngine

Stan: Not Initialized

Stan początkowy, przyjmowany przez obiekt klasy AiDetectionEngine bezpośrednio po jego utworzeniu. Silnik detekcji nie posiada jeszcze żadnych danych meczowych, żadnego skonfigurowanego źródła wideo ani załadowanych modeli detekcji. Obiekt oczekuje na dostarczenie protokołu meczowego przez kontroler BroadcastAnalysisController.

Stan: Initialization (stan złożony)

Stan złożony obejmujący cały proces przygotowania systemu do analizy transmisji. Zawiera trzy substany wykonywane sekwencyjnie. Z każdego z substanów możliwy jest powrót do stanu Not Initialized w przypadku wykrycia błędu: niepoprawnego protokołu (protocol invalid) lub błędu połączenia ze źródłem wideo (connection error). Wyjście ze stanu następuje po pomyślnym zakończeniu wszystkich trzech substanów — gdy spełniony jest warunek when(system is ready).

Podstan: Loading Match Data — *podstan stanu „Initialization”*

Obiekt klasy AIDetectionEngine pobiera i przetwarza protokół meczowy (obiekt klasy MatchProtocol) zawierający dane o meczu: identyfikator, datę, nazwę areny, składy drużyn oraz sędziów. Jeżeli protokół jest poprawny (protocol valid), system przechodzi do substanu Verifying Video Source. W przypadku nieprawidłowych lub niekompletnych danych (protocol invalid) następuje wyjście ze stanu złożonego i powrót do stanu Not Initialized.

Podstan: Verifying Video Source — *podstan stanu „Initialization”*

System weryfikuje dostępność i spójność źródła wideo. W zależności od jego typu wywoływana jest metoda verifyConnection() dla strumienia sieciowego (NetworkStream) lub checkIntegrity() dla pliku wideo (VideoFile). Pomyślna weryfikacja powoduje przejście do substanu Initializing AI Engine. Błąd połączenia (connection error) powoduje wyjście ze stanu złożonego i powrót do stanu Not Initialized.

Podstan: Initializing AI Engine — *podstan stanu „Initialization”*

Silnik detekcji AI jest inicjalizowany i kalibrowany. Pobierana jest pierwsza klatka inicjalizacyjna (wyslijKlatke()), która jest analizowana przez metodę analyzeInitializationFrame(). Następnie wywoływane są metody mapPlayers(), findReferee() oraz findBall(), które odpowiednio: przypisują wykrytych zawodników do rekordów protokołu meczowego, lokalizują sędziego i piłkę na boisku. Po zakończeniu kalibracji spełniony jest warunek when(system is ready) i obiekt opuszcza stan złożony Initialization.

Stan: Ready to Activate

Stan oczekiwania na uruchomienie analizy przez operatora. Silnik detekcji jest w pełni skonfigurowany i skalibrowany — modele są załadowane, zawodnicy zmapowani, a obiekty boiskowe zlokalizowane. Obiekt klasy AIDetectionEngine pozostaje w tym stanie do momentu naciśnięcia przez operatora przycisku START (wywołanie handleStartButton()), co powoduje przejście do stanu złożonego Active Analysis.

Stan: Active Analysis (stan złożony)

Stan złożony reprezentujący aktywną fazę analizy transmisji wideo w trakcie meczu siatkarskiego. System przetwarza kolejne klatki wideo w pętli, na przemian wykrywając obiekty na boisku i rejestrując zdarzenia meczowe. Stan jest opuszczany po usłyszeniu końcowego gwizdka sędziego (after(final whistle)).

Podstan: Processing Frame — *podstan stanu „Active Analysis”*

Pierwszy substan stanu Active Analysis. System pobiera kolejną klatkę wideo ze źródła (wyslijKlatke()) i przetwarza ją za pomocą silnika detekcji, wywołując metody findReferee() oraz findBall(). Po pomyślnym wykryciu obiektów na klatce (objects detected) system przechodzi do substanu Recording Event w celu zarejestrowania zaistniałego zdarzenia siatkarskiego.

Podstan: Recording Event — *podstan stanu „Active Analysis”*

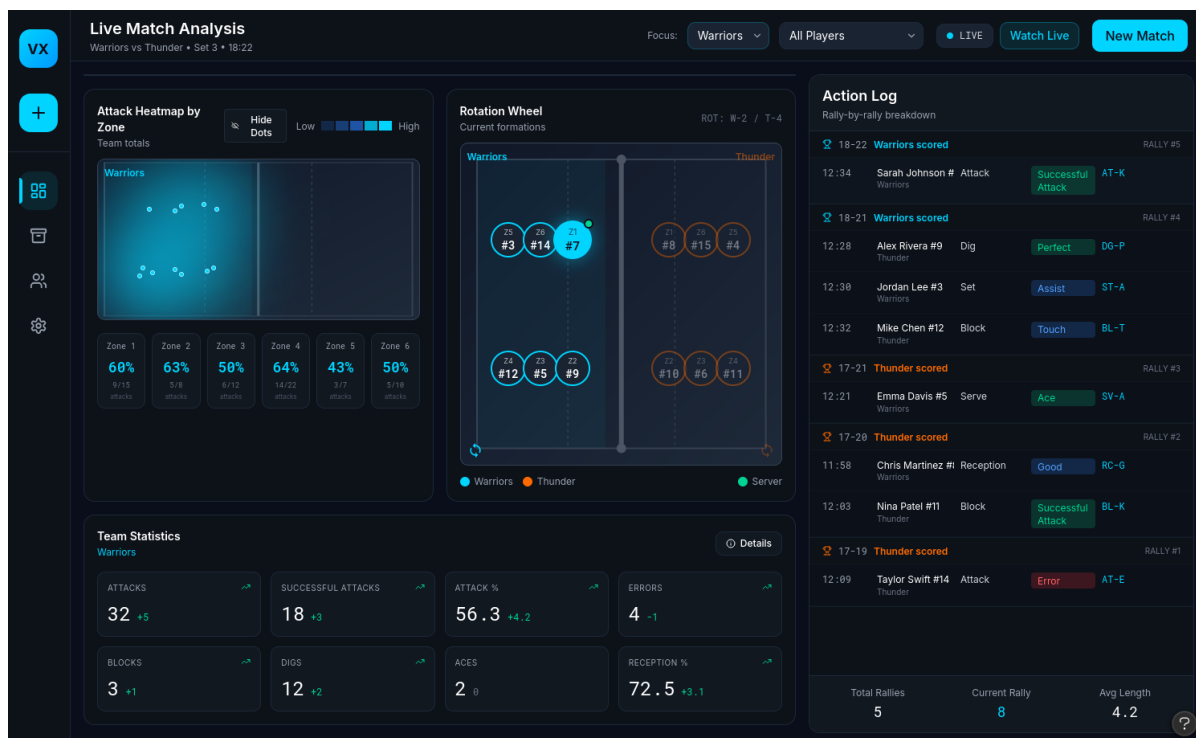
Na podstawie wyników detekcji moduł statystyk meczowych (MatchStatisticsModule) rejestruje zdarzenie siatkarskie poprzez wywołanie metody recordVolleyballEvent(). Rejestrowane mogą być zdarzenia takie jak zagrywka, atak, blok czy błąd. Po zakończeniu rejestracji, gdy dostarczona zostanie nowa klatka wideo (new Frame is send), system powraca do substanu Processing Frame, kontynuując pętlę analizy.

Stan: Finished

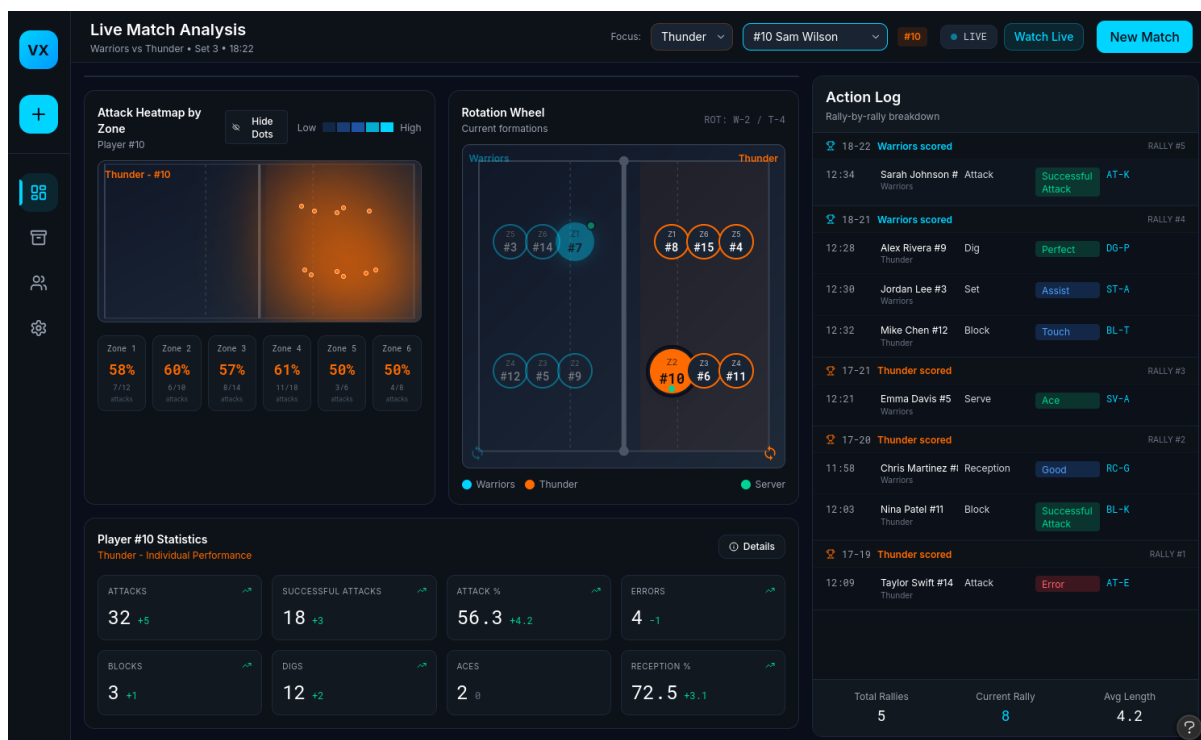
Stan przyjmowany przez obiekt po zakończeniu meczu (sygnał after(final whistle)). Automatyczna analiza transmisji wideo została zakończona — silnik detekcji AI nie przetwarza już kolejnych klatek. Jest to stan końcowy — po jego osiągnięciu obiekt klasy AIDetectionEngine przechodzi bezpośrednio do pseudostanu końcowego, co oznacza zakończenie cyklu życia obiektu.

Propozycje interfejsu użytkownika

Zaproponowano następujący układ menu głównego aplikacji. Wygenerowano go w narzędziu „figma”



Rysunek 16 Propozycja menu głównego z podglądem analizy meczu



Rysunek 17 Menu główne ze skupieniem na wybranego zawodnika

Zakładki umieszczone po lewej stronie okna umożliwiają zmianę widoku na:

- Widok główny analizy,
- Widok archiwum,
- Menu zarządzania drużynami,
- Ustawienia.

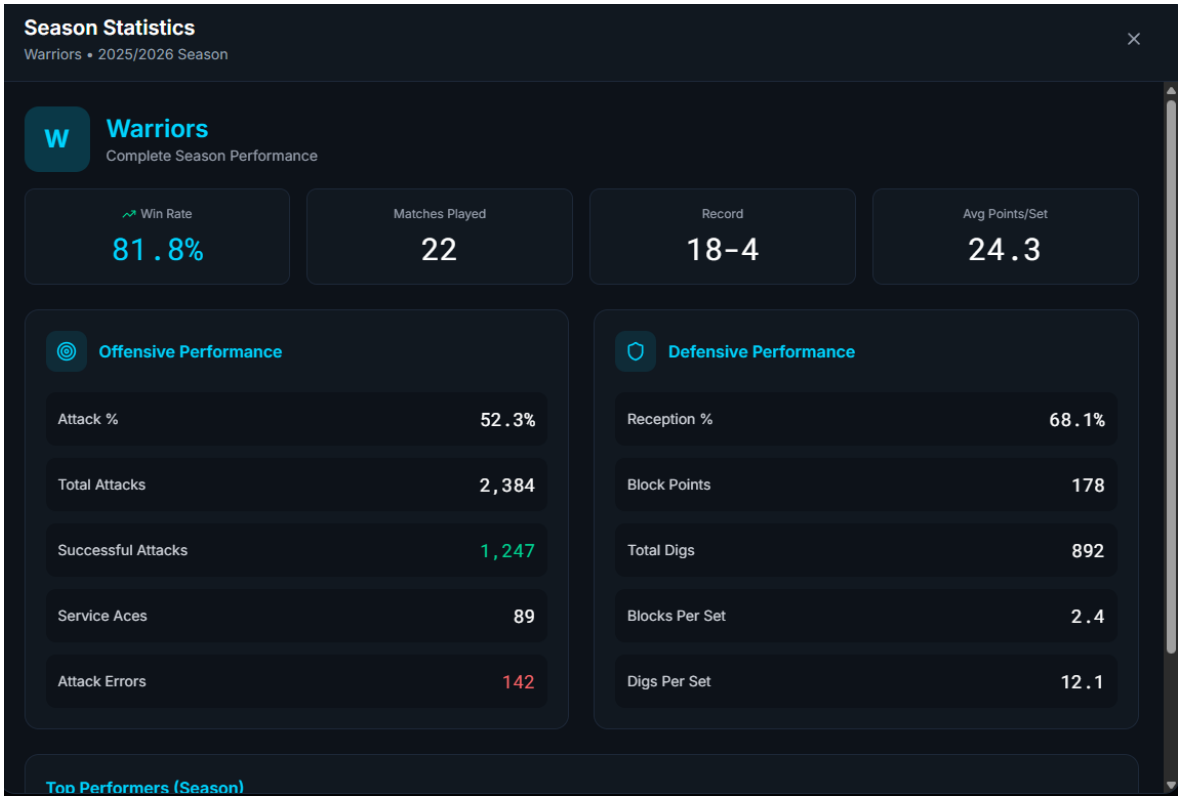
MATCH	DATE	TOURNAMENT	SCORE	DURATION	ACTIONS
Warriors vs Thunder 25-22, 20-25, 25-18, 23-25, 15-12	2026-04-28	National Championship	3-2	2h 14m	Details
Phoenix vs Eagles 25-18, 25-20, 25-19	2026-04-25	Regional Finals	3-0	1h 32m	Details
Warriors vs Phoenix 25-20, 22-25, 25-18, 25-21	2026-04-22	League Match	3-1	1h 58m	Details
Thunder vs Eagles 25-23, 20-25, 25-22, 18-25, 12-15	2026-04-20	National Championship	2-3	2h 28m	Details
Warriors vs Eagles 25-18, 25-18, 25-20	2026-04-18	League Match	3-0	1h 18m	Details

Rysunek 18 Ekran archiwum wcześniejszych meczy

Team	Win Rate	Record	Avg Pts
Warriors (12 players)	81.8%	18-4	24.3
Thunder (14 players)	68.2%	15-7	22.8
Phoenix (11 players)	54.5%	12-10	21.5
Eagles (13 players)	45.5%	10-12	20.2

Rysunek 19 Ekran zarządzania drużynami

W menu zarządzania drużynami możliwy jest również podgląd całosciowych statystyk danego zespołu. Po naciśnięciu przycisku *Details* przy interesującym nas wpisie ukazuje się poniższe okno.



Rysunek 20 Okno szczegółowych statystyk danego zespołu

Naciśnięcie przycisku *New Match* powoduje otwarcie okna przygotowania analizy nowego meczu.

New Match Setup
Step 1 of 3

Video Source

RTSP Stream
Live camera feed

Upload MP4
Recorded match

RTSP Stream URL

rtsp://camera.example.com:554/stream

Enter the RTSP URL for your camera feed

Cancel Continue

Rysunek 21 Ekran przygotowywania nowego meczu - możliwość skonfigurowania analizy obrazu z kamery na żywo

New Match Setup

Step 2 of 3

Team A

Warriors

Team B

Thunder

Tournament / League

e.g., National Championship 2026

Match Date

dd . mm . rrrr

Back

Continue

Rysunek 22 Ekran przygotowywania nowego meczu - wybór zespołów i turnieju

New Match Setup

Step 3 of 3



AI Calibration
Optimize AI models for this specific court and camera angle

☒

Ball tracking

☒

Player detection

☒

Action recognition

☐

Court boundary detection

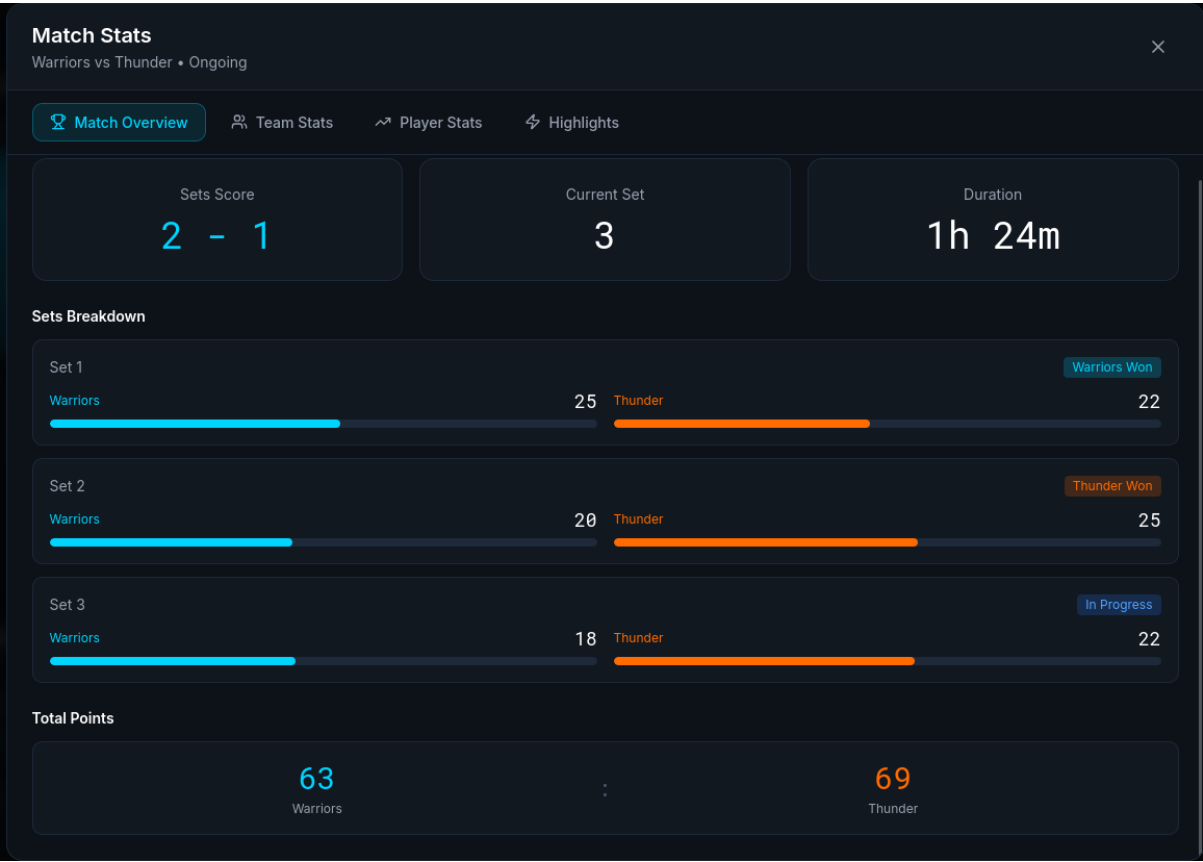
Tip: AI calibration runs automatically in the background. You can start analyzing immediately.

Back

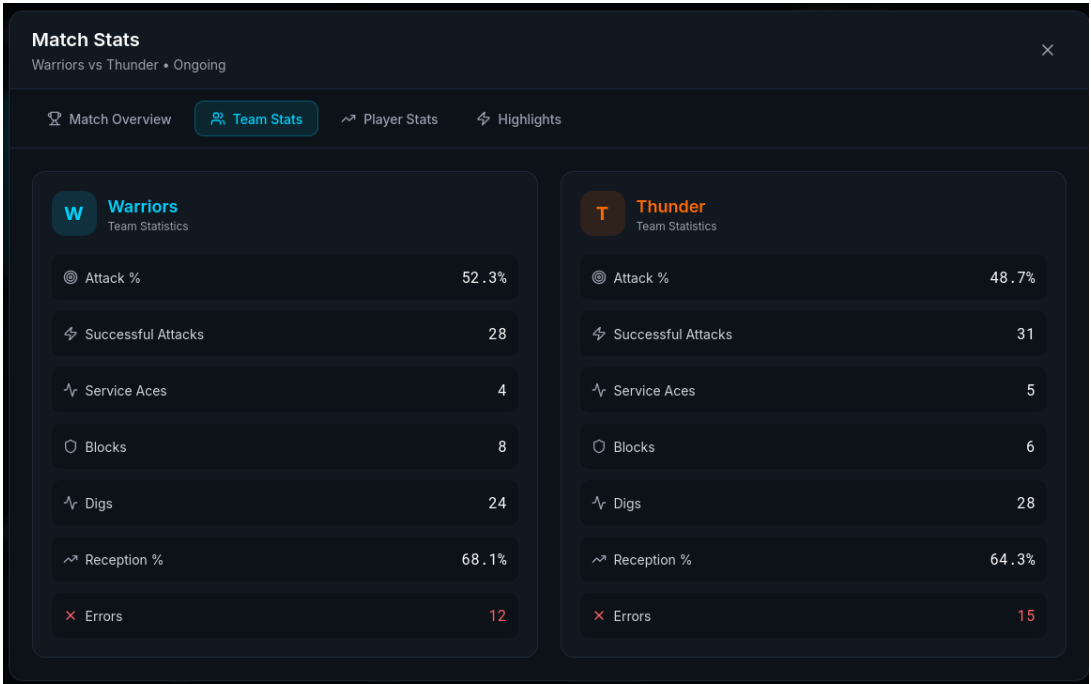
Start Match

Rysunek 23 Ekran przygotowania nowego meczu – opcje kalibracji AI

Po naciśnięciu przycisku *Details* na ekranie głównym przy statystykach zespołu ukazuje się okno ze szczegółowymi statystykami na temat danego meczu, gry drużyny i zawodników podczas aktualnego meczu oraz opis kluczowych wydarzeń w trakcie gry.



Rysunek 24 Okno szczegółowych statystyk na temat danego meczu



Rysunek 25 Okno szczegółowych statystyk obu zespołów w czasie trwania meczu

Match Stats

Warriors vs Thunder • Ongoing

Match Overview

Team Stats

Player Stats

Highlights

Warriors Players

#	PLAYER	POS	PTS	ATT	● SUCC	⬆ ACES	○ BLK	⬆ DIGS
7	Sarah Johnson OH	OH	18	32	18	2	3	12
12	Mike Chen MB	MB	12	20	11	0	4	3
5	Emma Davis S	S	8	15	7	1	0	18
9	Alex Rivera L	L	4	8	3	0	0	22
3	Jordan Lee OH	OH	15	28	14	1	1	8
14	Taylor Swift MB	MB	6	12	5	0	0	4

Thunder Players

#	PLAYER	POS	PTS	ATT	● SUCC	⬆ ACES	○ BLK	⬆ DIGS
8	Chris Martinez OH	OH	16	30	15	1	2	10
11	Nina Patel S	S	10	18	9	2	1	15

Rysunek 26 Okno szczegółowych statystyk poszczególnych zawodników w trakcie meczu

Match Stats

Warriors vs Thunder • Ongoing

Match Overview

Team Stats

Player Stats

Highlights

Match Highlights & Key Moments

⚡

Sarah Johnson #7

Warriors

Scored 5 consecutive aces

12:34

🛡

Maya Kim #4

Thunder

Triple block for the point

11:18

🏐

Jordan Lee #3

Warriors

Amazing dig save leading to score

10:05

🎯

Chris Martinez #8

Thunder

Perfect kill from Zone 4

08:42

Rysunek 27 Przegląd najważniejszych wydarzeń w trakcie meczu

Rejestracja nowej drużyny jest możliwa poprzez formularz dostępny poprzez wybranie przycisku *Add New Team* w menu zarządzania zespołami.

Strona 59 z 63

Register New Team ×

Enter team details and assign players to positions

Team Information

Team Name *

Coach Name (Optional)

Team Logo (Optional)

Player Roster ⊕ + Add Player

	Player Name *	Position *	Jersey # *
1	Enter name	Select position	#

* Required fields

Cancel Save Team

Rysunek 28 Okno dodawania nowej drużyny

6.) Wymagania нефunkcjonalne

Szacowana wielkość bazy danych przy 100 lig przy 30 000 zarejestrowanych użytkowników:

Na jedną drużynę przypada około:

1. ~21 użytkowników
 - a.) identyfikator = 4B
 - b.) login ~ średnio 15 znaków = $15 \cdot 4B = 60B$
 - c.) rola ~ średnio 12 znaków = $12 \cdot 4B = 48B$

W tym:

2. 20 zawodników
 - a.) numer = 4B
 - b.) Imię, Nazwisko ~ 36 znaków = $36 \cdot 4 = 144B$
 - c.) Kapitan ~ 1bit
 - d.) Statystyki ~ 6 atrybutów * 4B = 12B
3. 1 trener
 - a.) data_zatrudnienia ~8B
 - b.) Klub = referencja do obiektu 4B + 24B obiekt Klub z nazwą klubu = 28B

Razem: ~309B

Wymagany zasób do stworzenia kontrolera API zarządzający systemem:

Przykład (End Point do przydzielania sędziego):

1. AssignmentController ~ 24B
2. Assignment ~ 32B
3. Referee ~70B
4. RefereeRepository ~ 18B
5. NotificationService ~ 34B

Razem: ~178B

$309B \cdot 15 \text{ drużyn} = 4635 \text{ B}$ danych użytkowników

Szacowana liczba meczy ~10 000

$10000B \cdot 4635B = 46,35 \text{ MB}$ danych użytkowników
w każdym meczu

Analiza AI / Computer Vision

Przy czym zbiera się dane takie jak:

- pozycje zawodników
- trajektorie piłki
- wykryte akcje
- embeddingi AI

Przy założeniu, że średni mecz jest odtwarzany w 25 klatkach/s i trwa 2 godziny to wychodzi. 200–500 MB danych analitycznych

6.1. Oszacowanie wielkości bazy danych

6.2. Propozycja wymaganych czasów odpowiedzi

Wymagane czasy odpowiedzi

Określamy maksymalny całkowity czas od wysłania żądania przez aplikację użytkownika do wyświetlenia przez nią odpowiedzi. Zakładamy przy tym, że użytkownik posiada łącze o szybkości co najmniej 1 Mb/s i korzysta z serwisu na współczesnym urządzeniu pozwalającym komfortowo przeglądać zasoby internetowe. W przypadku wolniejszego łącza internetowego czas odpowiedzi będzie znacznie wydłużony.

- Utworzenie nowego konta użytkownika ~4.2s
- Pobranie listy sędziów ~ 1.0s
- Operacja przydziału sędziego do meczu ~ 2.1s
- Wysyłanie notyfikacji do sędziego o przydziale ~2.3s
- Potwierdzenie operacji i zapis do bazy danych ~2.7s
- Utworzenie drużyny siatkarskiej ~ 3.2s
- Pobranie wydarzeń podczas meczu ~1.1s
- Pobranie listy zawodników ~1.3s

6.3. Oszacowanie ilości i typów potrzebnych stanowisk pracy użytkowników systemu

7.) Technologie Informatyczne

Propozycja technologii informatycznych, które mogą zostać wykorzystane do realizacji systemu (sprzęt, oprogramowanie)

- diagram(-y) wdrożenia

(konkretne technologie ...)

8.) Analiza ryzyka i zagrożenia

wykaz przewidywanych zagrożeń (tylko zagrożenia specyficzne dla projektu):

- prawdopodobieństwo/szansa wystąpienia (np.: znikome, średnie, duże, bardzo duże)

- stopień szkodliwości w przypadku wystąpienia (np.: duży, średni, mały)

- propozycje metod zapobiegania danemu zagrożeniu

- plan awaryjny (sposób postępowania) w przypadku wystąpienia zagrożenia

9.) Plan Pracy

wyróżnione etapy (powiązane z konkretnymi częściami systemu, realizowanymi komponentami, ...) z podanym czasem trwania każdego etapu (dni robocze) - zależności pomiędzy etapami (np. co musi być zakończone przed rozpoczęciem kolejnego etapu) - alokacja zasobów ludzkich do realizacji poszczególnych etapów

10.) Kosztorys

(koszty projektu, oprogramowania, systemu, szkoleń, wdrożenia oraz konsultacji)

- w rozbiciu na mniejsze jednostki (etapy, podsystemy, moduły,...)

- warunki płatności, sposób odbioru

- może być wariantowy

11.) Aneksy

Umowa powierzenia przetwarzania danych osobowych (RODO)

Administratorem danych jest Polski Związek Piłki Siatkowej, ul. Puławska 300a, 02-819 Warszawa, NIP: 521-000-00-00, reprezentowany przez Prezesa Zarządu. Procesor zobowiązuje się przetwarzać dane osobowe wyłącznie w celu realizacji umowy głównej – dostarczenia, utrzymania i wsparcia systemu zarządzania kadrą siatkówkową. Na podstawie prawnej **Art. 6 ust. 1 lit. b RODO (umowa)** zbiera się dane osobowe zawodników, sędziów, trenerów oraz administratorów systemu.

Obowiązki Procesora

- ☐ Przetwarzać dane wyłącznie na udokumentowane polecenie ADO.
- ☐ Zapewnić poufność danych – zobowiązać pracowników do zachowania tajemnicy.
- ☐ Stosować odpowiednie środki techniczne i organizacyjne (szyfrowanie TLS 1.3, hasła bcrypt, MFA dla adminów).
- ☐ Nie powierzać danych podprocesorom bez uprzedniej pisemnej zgody ADO.
- ☐ Umożliwić ADO przeprowadzenie audytów i inspekcji.
- ☐ Usunąć lub zwrócić wszystkie dane po zakończeniu umowy (maks. 30 dni).
- ☐ Zgłaszać naruszenia ochrony danych do ADO w ciągu 24 godzin od wykrycia.

Środki bezpieczeństwa

W celu zapewnienia najwyższego poziomu bezpieczeństwa, zastosowano środki w celu sprywatyzowania transmisji danych. HTTPS protokół internetowy korzystający z dodatkowego szyfrowania. Nasze witryny są zabezpieczone za pomocą certyfikatów SSL.